Top 100+ Question

TECHNICAL INTERVIEW PREPARATION GUIDE

C#, .NET, SQL, OOPS, WEB API, ENTITY FRAMEWORK

Comprehensive Questions & Answers

CodewithIndu

OOPS&C#-Basics

Q1. What is C#? What is the difference between C# and .NET ?
Ans- C# (C-Sharp) is a modern, object-oriented programming language developed by Microsoft.
It is used to build secure, scalable, and high-performance applications such as web applications, desktop software, web APIs, mobile apps, cloud services, and games (Unity).

Key Points

1. Strongly-typed
2. Object-oriented
3. Component-based
4. Runs on the .NET platform
5. Designed for reliability and scalability

.NET is a complete software development platform that provides the runtime (CLR), libraries, tools, and frameworks needed to build and run applications written in C#, F#, or VB.NET.

Key Components of .NET

1. CLR (Common Language Runtime): Executes code
2. BCL (Base Class Library): Pre-built functions
3. ASP.NET Core: For web apps & APIs
4. Entity Framework Core: For data access
5. MAUI/Xamarin: For mobile apps

Real-World Example

Project Example: School Management System

- You write classes like Student, Teacher, Attendance in C#.
- You build your web application using ASP.NET Core (part of the .NET platform).
- The .NET runtime executes your C# code.

- Entity Framework Core handles the database operations.

So:

- C# = Code (Logic)
- .NET = Platform (Execution + Tools)

Q2. What is OOPS? What are the main concepts of OOPS?

Ans-**OOPS (Object-Oriented Programming System)** is a **programming model** that organizes software design around **objects** rather than functions and logic.

Is a way of code where we create objects (real- world models) to represent data and behaviour.

Its helps to write clean, reuseable and easy- to – maintain code.

An **object** = data (properties) + behaviour (methods).

Definition-

OOPS is a programming paradigm based on the concept of "objects" that helps create reusable, maintainable, scalable, and secure applications.

There are 4 main pillars of OOPS.

1. **Encapsulation**
2. **Abstraction**
3. **Inheritance**
4. **Polymorphism**

- **Encapsulation- "Data hiding"**
  Encapsulation means wrapping data (variables) and methods (functions) into a single unit called a class and protecting the data

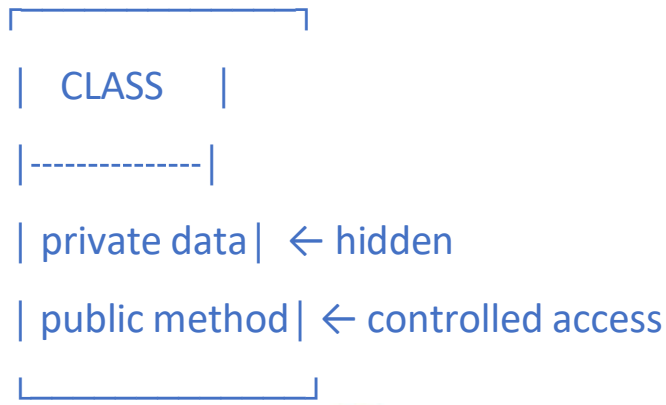
using access modifiers.

Example- A **capsule** contains medicine inside and protects it.

Similarly, a class hides internal data and exposes only required features.

Code-

```
class Student {
   private int age;     // hidden data
   public void SetAge (int a) {
      age = a;
   }
}
```



- **Abstraction-** Abstraction means showing only essential details and hiding the complex internal logic.
- **Inheritance-** Inheritance allows one class (child) to reuse the properties and methods of another class (parent).
- **Polymorphism-** Polymorphism means "many forms." It allows the same method to behave differently depending on the object.

Q3. What are the advantages of OOPS?

Ans- The advantages of OOPS (Object-Oriented Programming System) are the benefits it provides while designing software using objects, such as improved code organization, reusability, security, flexibility, and maintainability.

Top Advantages of OOPS

1. Reusability (Code Reuse)

OOPS allows developers to reuse classes and behaviour instead of writing the same code again.

Real-World Example

A **Car base class** can be reused to create Honda, BMW, Audi classes.

Benefit

Saves time, reduces errors, improves consistency.

2. Data Security (Encapsulation)

OOPS protects data using access modifiers like *private, protected, public*.

Real-World Example

A **Bank Account** hides its balance and exposes only deposit/withdraw methods.

Benefit

Prevents unauthorized access & improves safety.

3. Better Maintainability

OOPS divides the project into small, independent objects/classes.

Real-World Example

Changing the Student class does not force changes in Teacher class.

Benefit

Easier to maintain, update, and debug.

4. Abstraction (Hides Complexity)

OOPS hides unnecessary internal details and shows only essential features.

Real-World Example

A **Car's steering** hides complex engine mechanics.

Benefit

Cleaner interfaces, easier to understand and use.

5. Flexibility & Extensibility (Using Polymorphism)

OOPS allows methods to behave differently based on objects.

Real-World Example

A **Draw ()** method behaves differently for Circle, Rectangle, Triangle.

Benefit

Flexible design and easy extension.

6. Code Organization (Class & Object Structure)

OOPS organizes large projects into classes making the system structured.

Real-World Example

E-commerce app has separate classes: Product, Order, User, Cart.

Benefit

Clear architecture, modular design.

7. Improved Productivity

Using reusable classes, inheritance, and templates reduces development time.

Real-World Example

Developers can quickly create new screens/features using existing base classes.

Benefit

Software is built faster with fewer bugs.

Q4. What are the limitations of OOPS?

Ans- Limitations of OOPS are the disadvantages, challenges, or situations where object-oriented programming does not perform well or becomes unnecessarily complex.

Main Limitations of OOPS

1. **More Complex to Understand and Implement**

OOPS concepts (inheritance, polymorphism, abstraction) can be difficult for beginners.

Real-World Example

A simple script (like reading a file) becomes unnecessarily complicated when wrapped inside multiple classes.

2. Slower Performance Compared to Procedural Programming

OOPS uses dynamic dispatch, object creation, and garbage collection which make execution a bit slower.

Real-World Example

High-performance systems like OS kernels prefer C (procedural) over OOP.

3. Larger Memory Usage

Objects, class definitions, metadata, and runtime support require more memory.

Real-World Example

A simple calculator built using heavy OOP design may consume more RAM than needed.

---

4. Not Suitable for Small Programs

Small tasks (like reading/writing a file, printing logs) become complicated with OOPS structure.

Real-World Example

Creating classes like PrintUtility, FileReader, FileWriter for small scripts is overkill.

---

5. Risk of Over-Engineering (Too Much Abstraction)

Developers may create unnecessary classes, interfaces, or inheritance chains.

Real-World Example

Creating 7 layers of inheritance for a simple feature → makes debugging difficult.

6. Tight Coupling Due to Inheritance

Incorrect use of inheritance increases dependency between classes.

Real-World Example

If a parent class changes, all child classes must be updated.

---

7. Parallel/Multithreaded Programming Becomes Difficult

Managing multiple objects in parallel is complex and requires extra synchronization.

Real-World Example

Large banking systems with multiple transactions face concurrency challenges.

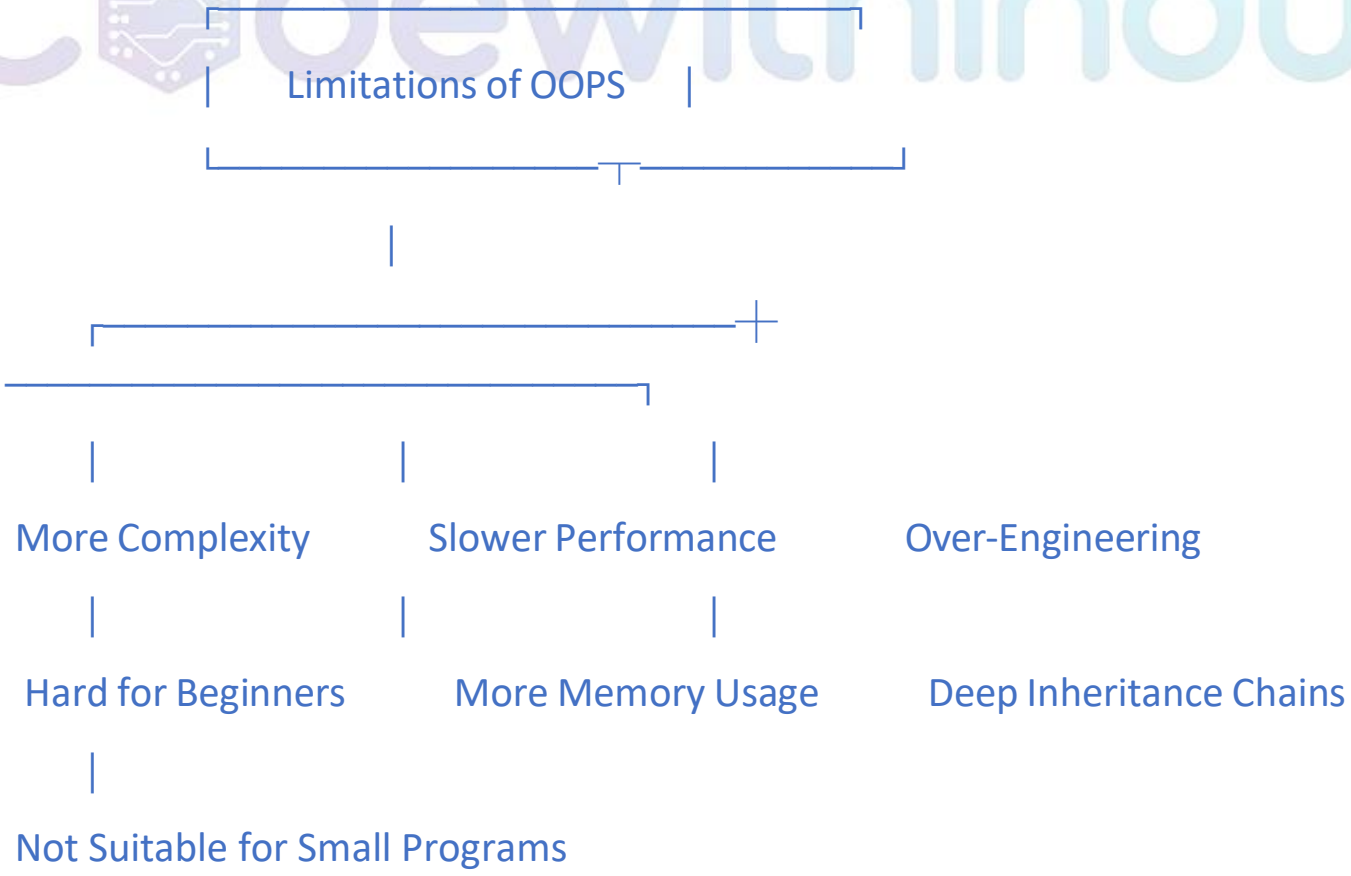
---

8. Not Ideal for Functional Requirements

Functional programming (like LINQ, Streams, Lambda) gives better performance in some scenarios.

Real-World Example

Data transformation pipelines are easier in functional than OOP style.



Q5. What are Classes and Objects?

Ans- **A class is a blueprint or template that defines the structure and behavior (properties and methods) of objects.**

It does not occupy memory until an object is created.

Example-

A Blueprint of a House

- The blueprint is not a real house
- It only describes how the house *should look*

Similarly,

- A **class** is a blueprint
- It describes how objects will look and behave

Code Example

```
class Student {
public string Name;
public int Age;

public void Study() {
    Console.WriteLine("Student is studying");
}
}
```

Codewithindu

Object

An object is an instance of a class. It represents a real entity created using the class blueprint.

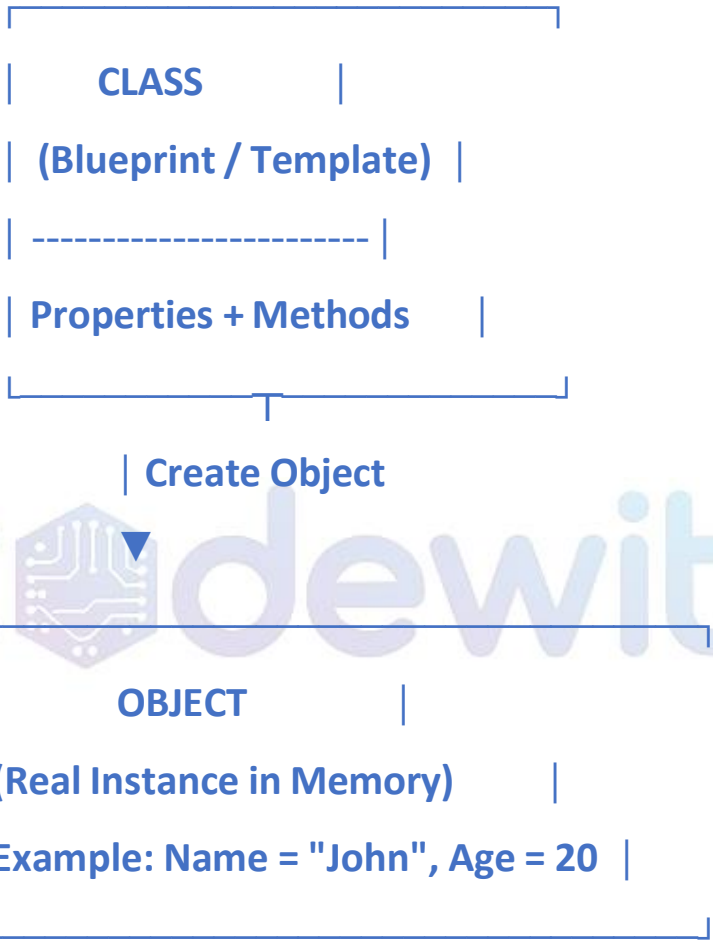
Objects occupy memory and can interact with each other.

Example-

Using the previous example:
Blueprint = Class
Actual House = Object

Code Example

```
Student s1 = new Student();   // s1 is an object
S1.Name = "John";
s1.Age = 20;
s1.Study();
```



Codewithindu

"A class is a blueprint or template that defines the properties and behaviors of an entity.
An object is an instance created from that class.
A class describes what the object will contain, while the object holds the actual data in memory.
For example, if 'Car' is a class, then the specific cars like BMW, Audi, and Honda are objects created from that class."

Q6. What are the types of classes in C#?

Ans- In C#, the main types of classes are:

1. Static Class
2. Sealed Class
3. Abstract Class
4. Partial Class
5. Concrete/Normal Class
6. Generic Class
7. Nested Class

Each class type serves a unique purpose such as inheritance control, code reusability, abstraction, or structure organization.

1. Concrete (Normal) Class

Definition - A normal class that can be instantiated (objects can be created).

Example

```
class Student {
   public string Name;
}
Student s = new Student();  // object created
```

2. Static Class

Definition - A class that **cannot be instantiated** and contains only **static members**.

Key Points

- No objects can be created
- Automatically sealed (cannot be inherited)
- Used for helper/utility functions

Example

```
static class MathHelper {
```

```
    public static int Add(int x, int y) => x + y;
}
```

3. Abstract Class

Definition

A class that **cannot be instantiated** and may contain **abstract methods**.

Purpose

To provide a **base structure** for derived classes.

Example

```
abstract class Animal {
    public abstract void Sound();
}
class Dog : Animal {
    public override void Sound() => Console.WriteLine("Bark");
}
```

Codewithindu

4. Sealed Class

Definition

A class that **cannot be inherited**.

Purpose

- Prevent modification
- Improve performance in some cases

Example

```
sealed class Payment { }
```

5. Partial Class

Definition

A class split across **multiple files**, combined at compile time.

Purpose

Useful for:

- Large classes
- Auto-generated + custom code separately (e.g., ASP.NET, EF)

Example

File 1:

```
partial class Employee {
    public string Name;
}
```

File 2:

```
partial class Employee {
    public void Work() { }
}
```

Codewithindu

6. Generic Class

Definition

Class that works with **different data types** without duplication.

Example

```
class Box<T> {
    public T Value;
}
Box<int> intBox = new Box<int>();
Box<string> strBox = new Box<string>();
```

7. Nested Class

Definition

A class declared **inside another class**.

Purpose

Used when a class is logically part of another.

Example

```
class Outer {
   class Inner {
      public void Show() { }
   }
}
```

“C# supports several types of classes such as Normal classes, Static classes, Abstract classes, Sealed classes, Partial classes, Generic classes, and Nested classes.
Each type serves a specific purpose—for example, static classes prevent object creation, abstract classes enforce base structure, sealed classes restrict inheritance, partial classes split code for maintainability, and generic classes help reusability with different data types.”

Q7. Is it possible to prevent object creation of a class in C#?

Ans- Yes, we can prevent object creation of a class in C# using any of the following ways:

1. Mark the class as static.
2. Make the constructor private.
3. Mark the class as abstract.

Each method prevents creation of objects in different scenarios.

1. Static Class

Definition

A static class **cannot be instantiated**.
Compiler does not allow creating objects from it.

Example

```
static class Logger
{
    public static void Log(string msg) { }
}

// Logger obj = new Logger();  ✕ Not allowed
```

When to Use

Utility classes like: MathHelper, Logger, ConfigManager.

Private Constructor

Definition

If the constructor is private, no object can be created from outside the class.

Example

```
class DatabaseConnection
{
    private DatabaseConnection() { }

    public static void Connect() { }
}

// DatabaseConnection db = new DatabaseConnection();  ✕ Not allowed
```

Real-World Use

Used in **Singleton Pattern**, Helper classes, Framework classes.

3.Abstract Class

Definition

Abstract class **cannot be instantiated**, but can be inherited.

Example

```
abstract class Shape
{
    public abstract void Draw();
}


// Shape s = new Shape(); ✕ Not allowed
```

Use Case

When you want to force child classes to give implementation.

Q8. What is Property?

Ans- A **Property** in C# is a **class member** that provides a **flexible way to read, write, or compute values** of private fields.
It acts like a **bridge** between **private data** and the **outside world**, ensuring **data encapsulation** and **validation**.

Properties use **get** and **set** accessors:

- **get** → returns the value
- **set** → assigns the value (with validation if needed)

Simple

Think of a property as a controlled door to a room (private field).
You can open it (get) or close it (set), but rules can be applied.

Example (Bank Account)

In real life, you cannot directly change your bank balance.
You update it through **controlled methods** (deposit, withdraw).

Same in C#:

We **don't expose balance directly**, we use a **Property** to control access.

Example:

```
public class BankAccount
{
    private double _balance; // Private field

    public double Balance
    {
        get { return _balance; }    // Read balance
        private set                 // Only class can set it
        {
            if (value >= 0)
                _balance = value;
        }
    }

    public void Deposit(double amount)
    {
        Balance = Balance + amount;
    }
```

Codewithindu

```
}
```

Q9. What is the difference between Property and Function?

Ans- **Property** A **Property** is a class member that provides **controlled access** to private fields using **get** and **set** accessors.
It is used to **store or retrieve data** and generally **should not perform heavy operations**.

◈ Function (Method)

A **Function/Method** is a **block of code** that performs **actions, computations, or business logic**.
It may take parameters, return values, and execute complex operations.

Simple Understanding

Concept	Easy Meaning
Property	Like a *door* to access private data (open/get, close/set).
Function	Like a *machine* that takes input, performs work, and outputs results.

Real-World Example (Bank ATM)

Property Example

Your **Bank Balance** is accessed safely:

- You can **read** it (get)
- You may **not directly set** it

```
public double Balance { get; private set; }
```

Balance is **data** → property is correct.

Function Example

Depositing or withdrawing money requires **logic**:

```
public void Deposit(double amount)
{
```

```
    Balance += amount;
}
```

Deposit is an **action** → function is correct.

Difference

Feature	Property	Function (Method)
Purpose	To expose data safely	To perform actions or logic
Accessors	Uses **get** and **set**	Uses **method body** with ()
Usage	Treated like a field	Called as an operation
Syntax	obj.Name	obj.CalculateAge()
Parameters	Usually no parameters	Can have multiple parameters
Return Value	Usually returns or sets a field	Can return anything or void
Workload	Should be simple and light	Can be complex and heavy
Usage in LINQ	Treated as data member	Not used as property
Memory	Can store data	Does not store data

Code Example (Clear and Simple)

Property

```
public string Name { get; set; }
```

Function

```
public string GetFullName()
{
    return FirstName + " " + LastName;}
```

Q10. What are Namespaces?

Ans- A **Namespace** in C# is a **logical grouping of related classes, interfaces, enums, and other code elements**.
It helps **organize code**, **avoid name conflicts**, and **manage large projects**.

Think of a namespace as a **folder** in your computer that contains related files.

Real-World Example

Imagine a large company with different departments:

- HR Department
- IT Department
- Finance Department

Each department has an employee named **"John"**—but they are in different departments, so there is **no conflict**.

Similarly:

- HR.John
- IT.John
- Finance.John

All can exist without conflict because they live in **different namespaces**.

C# Example

Creating a namespace

```
namespace MyApplication.HR
{
   public class Employee
   {
      public string Name { get; set; }
   }
}
```

Another namespace with same class name

```
namespace MyApplication.Finance
{
  public class Employee
```

```
    {
        public string Name { get; set; }
    }
}
```

Using a Namespace

```
using MyApplication.HR;


Employee emp = new Employee();
```

OOPS&C#-Inheritance, Abstraction, Encapsulation & Polymorphism

Codewithindu

Q11. What is Inheritance? When to use Inheritance?

Ans- **Inheritance** is an OOP concept in which one class (**child/derived class**) can acquire the properties and behaviors of another class (**parent/base class**).
It allows code **reuse**, **extensibility**, and creates a **hierarchical relationship** between classes.

The main keywords are:

- base (parent class)
- derived (child class)
- : (syntax for inheritance)

Simple

Inheritance means "child gets features of the parent".

Like in real life:

- A *Car* inherits from *Vehicle*

- A *Dog* inherits from *Animal*

The child has everything the parent has + its own features.

Real-World Example

Consider a Vehicle:

All vehicles have:

- Wheels
- Engine
- Move()

Now think of a Car:

- Car has wheels, engine, move() (from Vehicle)
- Plus: AC, Music System (its own features)

So Car inherits from Vehicle.

C# Example

Base Class (Parent)

```
public class Vehicle
{
    public int Wheels { get; set; }

    public void Move()
    {
        Console.WriteLine("Vehicle is moving");
    }
}
```

Derived Class (Child)

```
public class Car : Vehicle
{
    public void PlayMusic()
    {
        Console.WriteLine("Music is playing");
    }
}
```

Usage

```
Car car = new Car();
car.Move();       // From Vehicle
car.PlayMusic();  // From Car
```

Q12. What are the different types of Inheritance?

Ans - **Inheritance** is the mechanism by which one class acquires the properties and behavior of another class.
C# supports several types of inheritance but **not all**.

Types of Inheritance

C# supports these 5 common types:

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance (via Interfaces only) 5. Hybrid Inheritance (via Interfaces).

1. Single Inheritance

One child inherits from one parent.

Real-life example:

Dog → Animal
Employee → Person

Code:

```
class Animal
{
    public void Eat() { }
}


class Dog : Animal
{
    public void Bark() { }
}
```

Codewithindu

Diagram:

Animal

↑

Dog

---

2. Multilevel Inheritance

A child inherits from a parent, and another child inherits from that child.

Real-life:

Baby → Parent → GrandParent

Code:

```
class GrandParent { }

class Parent : GrandParent { }
```

```
class Child : Parent { }
```

Diagram:

```
GrandParent
    ↑
  Parent
    ↑
   Child
```

---

3. Hierarchical Inheritance

One parent has multiple children (siblings).

Real-life:

Teacher → MathTeacher, ScienceTeacher, EnglishTeacher

Code:

```
class Vehicle { }


class Car : Vehicle { }
class Bike : Vehicle { }
class Truck : Vehicle { }
```

Diagram:

```
      Vehicle
    ↑   ↑   ↑
   Car  Bike  Truck
```

---

4. Multiple Inheritance (Using Interfaces Only)

A class inherits from multiple interfaces.

 Real-life:

A SmartPhone:

- Calls (Phone)
- Takes Photos (Camera)
- Plays Music (Media Player)

Code:

```
interface ICamera { void Capture(); }

interface IMusic { void Play(); }


class SmartPhone : ICamera, IMusic
{
    public void Capture() { }

    public void Play() { }

}
```

Diagram:

```
ICamera    IMusic

  ↑        ↑

   \      /

     SmartPhone
```

---

5. Hybrid Inheritance (Combination using Interfaces)

A mix of:

Codewithindu

- Single
- Multilevel
- Multiple inheritance

C# supports hybrid inheritance but only through interfaces.

Example Structure:

IParent

↑

ParentClass

↑ ↑

Child1 IExtra

↑

Child2

Codewithindu

Not Supported in C#: Multiple Inheritance of Classes

```
class A { }

class B { }

class C : A, B  // ❌ Not allowed
```

Q13. Does C# support Multiple Inheritance? How to you implement?

Ans- **C# does NOT support multiple inheritance of classes.**

You **cannot** write:

```
class A { }

class B { }

class C : A, B   // ❌ Not allowed

{
```

}

This is because multiple inheritance creates **ambiguity** and **diamond problems**.

C# supports multiple inheritance through INTERFACES

(not through classes).

A class can inherit from **only one base class**, but it can **implement unlimited interfaces**.

Because two parent classes may have:

- Same method names
- Same property names
- Same constructors

This causes **ambiguity**.

Code Example of Multiple Inheritance Using Interfaces

```
interface ICamera
{
    void Capture();
}


interface IPlayer
{
    void PlayMusic();
}


class SmartPhone : ICamera, IPlayer
{
```

```
    public void Capture()
    {
        Console.WriteLine("Photo captured");
    }

    public void PlayMusic()
    {
        Console.WriteLine("Music playing");
    }
}
```

Usage:

```
SmartPhone phone = new SmartPhone();
phone.Capture();
phone.PlayMusic();
```

The **SmartPhone** gets features from **two parents**
This is multiple inheritance, but via **interfaces**

Real-World Explanation

A **Smartphone** has:

- Camera features
- Music player features
- Calling features

These come from different modules but combine into one device.

Similarly:

ICamera + IMusic + ICalling → SmartPhone

C# does not support multiple inheritance of classes, but it supports multiple inheritance through interfaces by allowing a class to implement multiple interfaces.

Q14. What is Abstraction? How to implement abstraction in real applications?

Ans-**Abstraction** is the OOP concept of **hiding complex internal details** and **showing only the necessary features** to the user.

It focuses on **what an object does**, not **how it does it**.

Abstraction means:
➞ "Don't show how things work internally, only show what the user needs."

Example:
You **use a mobile phone**, but you don't know how the internal circuits work.

Real-World Example (Very Easy)

Think about **driving a car**:

- You press **accelerator** → car speeds up.
- You press **brake** → car slows down.

But inside the car:

- Engine control
- Fuel combustion
- Transmission
- Brake hydraulics

All these complex processes are **hidden**.
You only interact with **simple controls**.

This is **abstraction**.

Simple Diagram

User

↓

[Car Dashboard]

(Accelerate, Brake, Start)

↓ (hidden)

Engine, Fuel System, Pistons, ECU

User sees simple features.
Complex things stay hidden.

C# Code Example of Abstraction (Using Abstract Class)

Abstract Class

```
public abstract class Payment
{
    public abstract void MakePayment();  // Hide internal steps
}
```

Concrete Classes

```
public class CreditCardPayment : Payment
{
    public override void MakePayment()
    {
        Console.WriteLine("Payment done using Credit Card");
    }
}
```

Codewithindu

```
public class UPIPayment : Payment
{
    public override void MakePayment()
    {
        Console.WriteLine("Payment done using UPI");
    }
}
```

Implement Abstraction in Real Applications

Abstraction is used everywhere in software development.

1. Using Abstract Classes

Used when:

- You want to define a common template.
- You want some methods to be **implemented by child classes**.

Example in real apps:

- Payment System
- Notification System
- Logging System

2. Using Interfaces

Interfaces hide logic and show only **methods** the developer must implement.

Example:

- IRepository in Repositories
- ILogger in Logging
- IEmailService in Email sending
- IDbConnection in ADO.NET

3. Using Encapsulation + Abstraction

Use private fields + public methods to hide internal logic.

Example:

```
class BankAccount
{
    private decimal balance;


    public void Deposit(decimal amount)
    {
        // internal logic hidden
        balance += amount;
    }
}
```

User doesn't see calculation logic → abstraction.

---

4. APIs (Real Application Example)

When you call:

POST /api/payment/process

You don't know:

- Which bank is used
- How verification happens
- How encryption works

The API hides all internal implementation → **abstraction**.

---

5. Layers in Architecture

In real projects (Onion, Clean, N-Tier), abstraction is used:

- UI layer does not know service details
- Service layer does not know repository details
- Repository hides database queries

This makes applications maintainable and scalable.

Abstraction means showing only essential features and hiding internal details. It is implemented using abstract classes, interfaces, and layered architecture in real applications.

Q15. What is Encapsulation? How to implement encapsulation in real applications?

Ans-**Encapsulation is the process of wrapping data (variables) and methods (functions) into a single unit and hiding the internal details from the outside world.**

It protects data by **controlling access** through **getters & setters** (properties), not allowing direct modification.

Example

Think of a **mobile phone**:

- You see a beautifully designed home screen (public interface).
- You do **not** see how the motherboard, RAM, or battery works (private details).
- You just tap buttons → internal system handles it safely.

This is **Encapsulation** — exposing only what is necessary and hiding internal complexity.

Technical Definition

Encapsulation =**Data Hiding + Controlled Access + Security**

In C#, we achieve encapsulation using:

- **Private fields**
- **Public properties (get/set)**
- **Access Modifiers (private, public, protected, internal)**

Real-World Example in Applications

Example: Bank Account System

We never want someone to directly change the account balance like:

account.balance = 1_000_000; // Dangerous!

So we use encapsulation:

```
public class BankAccount
{
    private decimal _balance;     // 🔐 Hidden (private)

    public decimal Balance        // 🛠 Controlled access (public property)
    {
        get { return _balance; }  // Read allowed
        private set { _balance = value; }  // Update only within class
    }

    public void Deposit(decimal amount)    // Public method (safe operation)
    {
        if (amount > 0)
            Balance += amount;
    }

    public void Withdraw(decimal amount)  // Controlled withdraw
    {
```

Codewithindu

```
        if (amount > 0 && amount <= Balance)
            Balance -= amount;
    }
}
```

Benefits:

1. No one can directly modify _balance.
2. Only **safe methods** (Deposit, Withdraw) update the value.
3. Prevents invalid states (negative balance).

This is perfect encapsulation!

Another Example: Student Management System

You never expose fields directly:

```
public class Student
{
    private int _age;


    public int Age
    {
        get { return _age; }
        set
        {
            if (value >= 5 && value <= 100)    // Validation
                _age = value;
        }
    }
```

Codewithindu

```
}
```

Here:

- _age is hidden
- Age property is used to safely set the value

Q16. What is the difference between Abstraction and Encapsulation?

Ans-**One-Line Difference**

- **Abstraction = Hiding *implementation details*** and showing **only essential features**.
- **Encapsulation = Hiding *data* and protecting it through access control**

Concept	Meaning	Purpose	How it Works
Abstraction	Shows *what* the object does	Reduce complexity	Achieved using abstract classes, interfaces
Encapsulation	Hides *how* the object stores/handles data	Protect data	Achieved using private fields + public getters/setters

Real-Life Example

Mobile Phone Example

- **Abstraction**: You press a button to take a photo.
  You know *what* it does, not *how* the camera hardware works inside.
- **Encapsulation**: Your phone hides internal data like memory, battery level control, system files — you cannot directly change them.

Real-World Software Example

Abstraction Example (What):

```
public interface IPayment

{

    void Pay();
```

```
}
```

This tells **what** the method will do—not how.

Encapsulation Example :

```
public class BankAccount
{
    private decimal _balance; // Hidden (data)

    public decimal Balance
    {
        get { return _balance; }
        private set { _balance = value; }
    }

    public void Deposit(decimal amount)
    {
        Balance += amount;  // Controlled access
    }
}
```

- _balance is **protected** → Encapsulation
- Only required methods are shown → Abstraction

Abstraction focuses on hiding implementation details and showing only the required features.
Encapsulation focuses on hiding data and controlling access to it through getters and setters.

Codewithindu

Abstraction is achieved using interfaces and abstract classes, while encapsulation is achieved using private fields and public properties.

Comparison

Feature	**Abstraction**	**Encapsulation**
Definition	Hides implementation	Hides data
Focus	*What* to do	*How* to protect data
Purpose	Reduce complexity	Increase security
Achieved With	Interfaces & Abstract classes	Classes, access modifiers (private, public)
Example	IPayment.Pay()	private balance; public Deposit();
Visibility	Shows essential parts	Hides sensitive parts

Q17. What is Polymorphism and what are its types? When to use polymorphism?

Ans-**Polymorphism** means **one action behaving differently in different situations**.

codewithindu

Poly = many
Morph = forms

So, **Polymorphism = Many Forms**

Polymorphism in C# allows a single method, function, or object to behave differently based on the context.
It increases **flexibility**, **reusability**, and **maintainability**.

Real-World Example

Example: "Send Message" button

On different apps:

- WhatsApp → sends text
- Gmail → sends email
- Instagram → sends DM

Same action → different outputs
This is polymorphism.

Software Example

A Print() method:

- If object = PDF → print PDF
- If object = Image → print Image
- If object = Text → print text

Same method name, different behavior.

Types of Polymorphism in C#

There are **two main types**:

Compile-time Polymorphism (Static Binding / Early Binding)

Method Overloading

Same method name, different parameters.

Example:

```
public class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public double Add(double a, double b)
    {
        return a + b;
    }
}
```

Codewithindu

Use When: You want different ways to call the same method.

Runtime Polymorphism (Dynamic Binding / Late Binding)

Method Overriding

Same method name and signature, but defined in **child class**.

Needs:

- Inheritance
- virtual keyword in base class
- override keyword in child class

Example:

```
public class Animal
{
    public virtual void Sound()
    {
        Console.WriteLine("Animal makes sound");
    }}
public class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Dog barks");
    }
}
```

Codewithindu

```
public class Cat : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Cat meows");
    }
}
```

Output:

- Dog → Bark
- Cat → Meow

Same method → different results

When to Use Polymorphism?

Use polymorphism when:

1. You want common behavior but different implementations

Example: Payment gateways

- PayPal
- RazorPay
- Stripe

Each has its own process, but method = ProcessPayment()

---

2. You want to write flexible and scalable code

Example:

Animal a = new Dog();

Codewithindu

a.Sound();

You can easily switch Dog → Cat → Cow without changing your code.

3. You want to follow Open/Closed Principle (OOP SOLID)

"Open for extension, closed for modification."

4. You want to reduce if/else or switch mess

Instead of:

```
if(type == "dog") { bark(); }

else if(type == "cat") { meow(); }
```

Use polymorphism.

Polymorphism allows one method or object to take many forms.
It has two types: Compile-time polymorphism (method overloading) and Runtime polymorphism (method overriding).
We use polymorphism to write flexible, scalable code where different classes provide different implementations for the same method.

Q18. What is Method Overloading? In how many ways a method can be overloaded?

Ans-Method Overloading means creating **multiple methods** with the **same name** but **different parameters** in the **same class**.

Same method name

Different form (parameters)

Definition

Method Overloading in C# is a type of **Compile-Time Polymorphism** where multiple methods share the **same name** but differ by:

- Number of parameters
- Type of parameters
- Order of parameters

This allows the same method name to perform **similar actions** in different ways.

Easy Real-World Example

Imagine a **"Book Ticket"** button:

- Book ticket with Name
- Book ticket with Name + Seat number
- Book ticket with Name + Seat + Meal type

Same action → different inputs → method overloading.

C# Example

```
public class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public double Add(double a, double b)
    {
        return a + b;
    }


    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
}
```

Codewithindu

Here:
✓ same method name = Add
✓ different parameters → valid overloading

In How Many Ways Can a Method Be Overloaded?

A method can be overloaded in **3 ways**:

1- **By Changing the Number of Parameters**

void Print(int a)

void Print(int a, int b)

2- By Changing the Data Type of Parameters

void Print(int a)

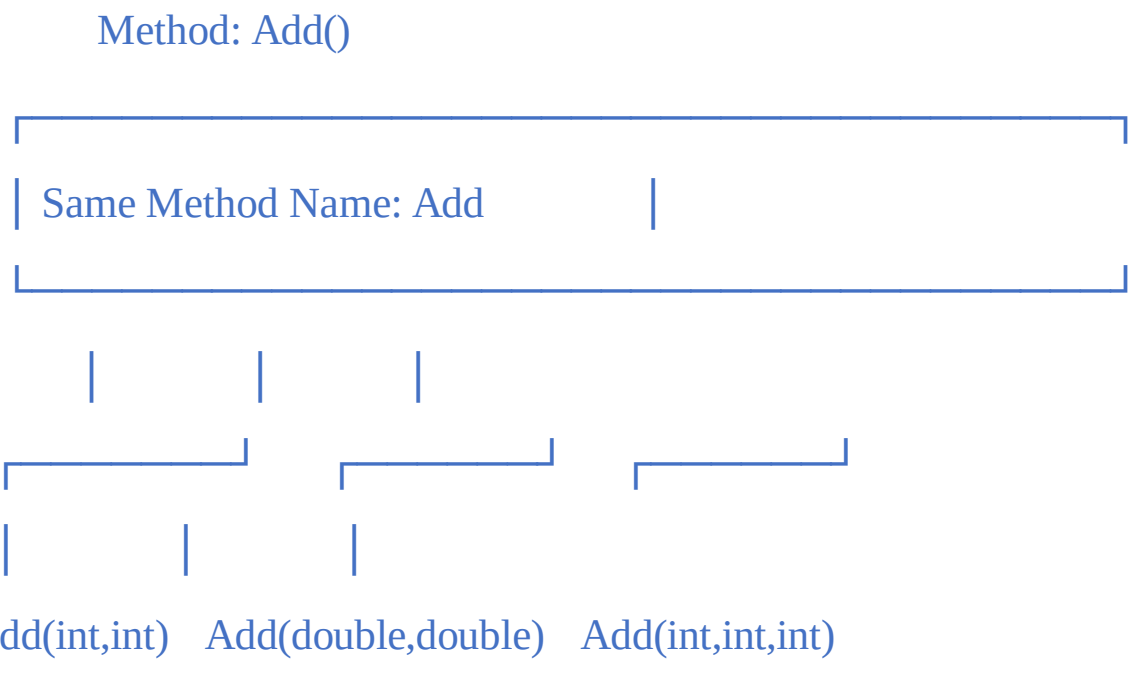
void Print(string a)

3- By Changing the Order of Parameters

void Print(int a, string b)

void Print(string a, int b)

Diagram-



When Should You Use Method Overloading?

Use it when:

✓ You want the **same action** with **different inputs**
✓ You want better readability
✓ You want cleaner code
✓ You want to avoid multiple method names like:
Add1()
Add2()
Add3() (not professional)

Method Overloading is compile-time polymorphism where multiple methods have the same name but different parameter lists.
A method can be overloaded by changing (1) number of parameters, (2) type of parameters, or (3) order of parameters.

Q19. What is the Difference Between Overloading and Overriding?

Ans-**Method Overloading**

Same method name, **different parameters**, **same class**.
→ *Compile-time polymorphism*

Method Overriding

Same method name, **same parameters**, **different class (inheritance)**.
→ *Runtime polymorphism*

Real-World Example

Overloading Example (Same action, different ways)

Restaurant Order:

- Order("Burger")
- Order("Burger", quantity)
- Order("Burger", quantity, spice level)

Same action "Order", different input.

Overriding Example (Different behavior)

A base class **Vehicle** has Start().
Different child classes override behavior:

- Car → Start using key
- Bike → Start using self-start
- Electric Car → Start using button

Same method, different behavior.

Code Example

Overloading Example

```
public class Calculator
{
    public int Add(int a, int b)
    {
        return a + b;
    }

    public double Add(double a, double b)
    {
        return a + b;
    }

    public int Add(int a, int b, int c)
    {
        return a + b + c;
    }
```

Codewithindu

```
}
```

Overriding Example

```
public class Animal
{
    public virtual void Sound()
    {
        Console.WriteLine("Animal makes sound");
    }
}

public class Dog : Animal
{
    public override void Sound()
    {
        Console.WriteLine("Dog barks");
    }
}
```

Difference

Feature	Overloading	Overriding
Meaning	Same method name, different parameters	Same method name & parameters, different implementation
Type	Compile-time polymorphism	Runtime polymorphism
Happens In	Same class	Parent–child classes

Codewithindu

Requires Inheritance?	✕ No	✓ Yes
Keywords Used	None	virtual, override
Parameter Change	Must change	Must NOT change
Purpose	Flexibility – different ways to call same method	Custom behavior in derived class
Binding	Early binding	Late binding

OOPS&C#-Abstract Class & Interface

Q20. What is the Difference Between an Abstract Class and an Interface?

Ans-Abstract Class

A class that **cannot be instantiated** and may contain **abstract (unimplemented)** and **non-abstract (implemented)** methods.

Interface

A contract that contains **only definitions** (no implementation).
Classes must **implement all methods** of the interface.

Definition

- An **abstract class** is used to define a **common base** with shared code.
- An **interface** defines a **contract/standard** that multiple unrelated classes must follow.

Real-World Example

Abstract Class (Base Model with Common Features)

Car has:

- Steering → available in all cars (implemented)
- StartEngine() → each car starts differently (abstract)

So:

```
abstract class Car
{
    public void Steering() { }       // implemented
    public abstract void Start();    // not implemented
}
```

Interface (Rules / Contracts)

Example:

```
interface IPayment
{
    void Pay();
    void Refund();
}
```

Interface = “Rules only, no implementation.”

Code Comparison

Abstract Class Example

```
public abstract class Animal
{
    public void Eat()
    {
        Console.WriteLine("Eating...");
    }

    public abstract void Sound();
```

Codewithindu

```
}
```

Interface Example

```
public interface IAnimal
{
    void Eat();
    void Sound();
}
```

Difference Table

Feature	Abstract Class	Interface
Object Creation	✕ Not possible	✕ Not possible
Methods	Can have **abstract + normal (implemented)** methods	Only **abstract methods** (until C# 8)
Access Modifiers	Can have **private, protected, public**	All methods are **public** by default
Variables	Can have **fields** (variables)	Cannot have fields (only properties/constants)
Multiple Inheritance	✕ Not supported	✓ Supported — a class can implement many interfaces
Constructor	✓ Can have constructors	✕ Cannot have constructors
Purpose	When classes are **related** and share common logic	When you want a **contract** for unrelated classes
Inheritance	class : abstractClass	class : IInterface
Implementation	Some code already written	No code written (only signatures)
Speed	Slightly faster	Slightly slower

An abstract class can contain both abstract and non-abstract methods, fields, and constructors.
An interface contains only method signatures and acts as a contract.
Classes can inherit only one abstract class but can implement multiple interfaces.

Use abstract class for shared logic and interface for defining common rules across unrelated classes.

Q21. When to Use Interface and When to Use Abstract Class in Real Applications?

Ans-Simple Rule

Use INTERFACE when you need a *contract* (rules).

Use ABSTRACT CLASS when you need a *common base* (shared logic).

- Interface defines what must be done.
- Abstract class defines what + some part of how it should be done.

Q29. Why to create Interfaces in real applications?

Ans- Interfaces define a contract that classes must follow. They ensure consistency, loose coupling, flexibility, and testability across the application.

Benefit	**Reason**
Loose Coupling	Less dependency on concrete classes
Testability	Mocks, stubs required for unit testing
Flexibility	Replace implementations without breaking code
Multiple Inheritance	A class can implement multiple interfaces
Clean Architecture	Services, repositories always interface-based
DI Support	Frameworks require interfaces
Scalability	Easily add new features or modules
Maintainability	Changes in class don't affect consumers

We create interfaces to achieve loose coupling, increase flexibility, support dependency injection, enable unit testing, enforce consistency, and allow multiple implementations without changing existing code. Interfaces help build scalable, maintainable, and clean architecture in real enterprise applications.

Q23. Can we define the body of interface methods?

Ans-In C#, an **interface** is a contract that contains **method signatures**, not implementation.

But starting from C# 8.0, Microsoft introduced:

1. **Default Interface Methods** – can have a method body
2. **Static Methods in Interface** – can have implementation

Traditional abstract methods in an interface still cannot have a body.

Real-World Example

Example: Payment Gateway System

You have multiple payment methods:

- Google Pay
- PhonePe
- Paytm
- Credit Card

All must follow a common method:

```
void ProcessPayment();
```

But you also want a default method:

```
void LogTransaction()
```

Every payment gateway should log the transaction in the same way.

Code Example

```
public interface IPayment
{
    void ProcessPayment(); // Cannot have body


    // Default method – Allowed (C# 8+)
    public void LogTransaction()
    {
        Console.WriteLine("Transaction logged successfully.");
```

```
    }

    // Static method – Allowed
    public static void ShowPaymentRules()
    {
        Console.WriteLine("Payments are secure.");
    }
}
```

Implementation

```
public class GooglePay : IPayment
{
    public void ProcessPayment()
    {
        Console.WriteLine("Google Pay Payment Processed.");
    }
}
```

Q24. Can an Interface have a Constructor?

Ans- An **interface** is a *contract* that defines **what** a class should do — not *how* to do it.
Therefore, **interfaces cannot contain:**

- Constructors ✕
- Instance fields ✕
- Instance state ✕

Because an interface **cannot be instantiated**, there is no need for a constructor.

Simple Explanation

Interfaces cannot create objects.
Only classes/structs that implement the interface can be instantiated.

1. **Interfaces cannot have state.**
   Constructors are used to initialize state → but interfaces cannot hold state.
2. **Interfaces define behavior only**
   No memory allocation → No object → No constructor required.

Example: Online Payment Contract

You create an interface:

```
public interface IPayment
{
    void ProcessPayment();
}
```

It makes no sense to do this:

```
IPayment p = new IPayment();   // ❌ IMPOSSIBLE
```

Because **IPayment** is just a contract, not an object.

So since we cannot create an interface object → we don't need a constructor.

Q25. What is the difference between Abstraction and Abstract Class?

Ans-Abstraction → A *concept* of hiding internal details and showing only essential features.

Abstract Class → A *way to implement abstraction* using a special class with abstract and non-abstract members.

Abstraction (Concept)

Abstraction is an **OOP principle** that hides complex internal logic and exposes only necessary information.

It focuses on **what** an object does, not **how** it does it.

Example:
A *mobile phone* hides internal circuits and shows only buttons/UI to the user.

2️⃣Abstract Class (Implementation Tool)

An abstract class is a **special class** in C# that:

- Cannot be instantiated
- Can contain **abstract methods** (no body)
- Can also contain **normal methods with implementation**
- Is used to provide *partial implementation*
- Helps implement abstraction

Example: ATM Machine

Abstraction :

- User sees:
  - Withdraw Money
  - Check Balance
  - Print Receipt
- Hidden details:
  - Database operations
  - Network calls
  - Account validation

Abstract Class (Code Implementation):

```
public abstract class ATM
{
    public abstract void Withdraw();
    public abstract void CheckBalance();

    public void PrintReceipt()
    {
        Console.WriteLine("Receipt printed...");
```

```
    }
}
```

Here:

- Withdraw() and CheckBalance() → abstract methods (no body)
- PrintReceipt() → implemented method

This class is **implementing abstraction**.

Differences

Feature	Abstraction	Abstract Class
Type	Concept	Code structure
Purpose	Hide details	Achieve abstraction
Instantiation	Not applicable	Cannot be instantiated
Contains implementation?	No	Yes (partial)
Usage	High-level design	Inheritance & code reuse

Abstraction is an OOP concept that hides internal details and exposes only essential features.
An abstract class is a special class in C# used to implement abstraction. Abstraction is 'what to show', whereas an abstract class partially defines 'how to show it'.

Q26. What are Access Specifiers?

Ans-In C#, **Access Specifiers** (also called *Access Modifiers*) are keywords that define the **visibility** and **access level** of classes, methods, variables, and other members.

They control **who can access what** in your application.

They help in:

- **Security**
- **Encapsulation**

- **Maintaining clean architecture**

Access Specifiers Are Important

Think of a **house**:

- Some rooms are public (living room)
- Some are private (bedroom)
- Some are protected (family only)
- Some are internal (store room inside house only)

Exactly the same in C#.

Types of Access Specifiers (C#) □

public

- Accessible from **anywhere**.
- Least restrictive.

Example:

```
public class Student
{
    public string Name;
}
```

□private

- Accessible **within the same class only**.
- Most restrictive.

Example:

```
public class Account
{
    private double balance;  // Only Account class can access
}
```

Codewithindu

protected

- Accessible in:
  - Same class
  - Derived (child) classes

Example:

```
public class Animal
{
    protected void Eat() { }
}


public class Dog : Animal
{
    void Test()
    {
        Eat(); // Allowed
    }
}
```

internal

- Accessible **within the same project/assembly** only.

Example:

```
internal class Helper { }
```

protected internal

Combination of:

Codewithindu

- protected
- internal

Accessible in:

- same assembly
- AND
- derived classes (even if in another assembly)

6️⃣ private protected

Combination of:

- private
- protected

Accessible in:

- same class
- derived classes **but only in the same assembly**

Banking App Example:

```
public class BankAccount
{
    private double balance;          // sensitive → private
    protected void CalculateInterest() { }  // only child classes
    internal string BankCode;        // only within same project
    public string AccountHolderName;  // visible everywhere
}
```

Access specifiers in C# are keywords that define the accessibility level of classes and their members. They control which parts of code can access a particular member. C# provides six access specifiers: public, private, protected, internal, protected internal, and private protected.

Q27What is the difference between “String” and “StringBuilder”?

Ans-**String (Immutable)**

In C#, **String** objects are **immutable**.
This means once a string is created, **any modification creates a NEW string object** in memory.

```
string name = "John";

name = name + " Doe"; // Creates NEW string object
```

2️⃣StringBuilder (Mutable)

StringBuilder allows **modifying the same object** without creating new objects repeatedly.

Used for:

- Large text modifications
- Loops
- Performance-heavy string operations

```
StringBuilder sb = new StringBuilder("John");

sb.Append(" Doe"); // Same object modified
```

Imagine a Whiteboard vs Paper

Concept	Real-World Example
String	Writing on paper → You cannot erase → You must take NEW paper every time
StringBuilder	Writing on whiteboard → You can modify same board again & again

Difference Table

Feature	String	StringBuilder
Mutability	Immutable	Mutable
Memory usage	High (creates new objects)	Low (modifies same object)
Performance	Slow for repeated changes	Fast for repeated changes
Namespace	System	System.Text
Use case	Small or static text	Large or frequently modified text

Feature	String	StringBuilder
Thread-safe	Yes	No (Not thread-safe by default)

Q28. What are the Loop types in C#? When to use what in real applications?

Ans-A **loop** in C# is a control structure that allows you to **repeat a block of code** multiple times until a condition is met.

Types of Loops in C#

C# provides **5 main loop types**:

1. **for loop**
2. **foreach loop**
3. **while loop**
4. **do-while loop**
5. **nested loops** (loops inside loops → combination of above)

1. for Loop

When you know how many times you have to repeat.

Syntax:

```
for(int i = 0; i < 10; i++)
{
   Console.WriteLine(i);
}
```

Real-World Example:

- Display top 10 products
- Run loop for 12 months
- Send SMS to first 100 customers

When to Use:

- Fixed count iterations
- Index-based operations
- Working with arrays using index

2. foreach Loop

Collections, lists, arrays, when you want to read each item.

Syntax:

```
foreach(var item in employees)
{
    Console.WriteLine(item.Name);
}
```

Real-World Example:

- Loop through list of students from DB
- Send emails to all users
- Display all items in a shopping cart

When to Use:

- When you don't need index
- When you want safe, clean iteration
- Perfect for database & API response lists

3. while Loop

Repeat until a condition becomes false, but you don't know how many times it will run.

Syntax:

```
while(balance > 0)
{
    balance -= 100;
}
```

Real-World Example:

- Keep reading data from API until data ends

- Keep showing ATM menu until user exits
- Retry logic until request succeeds

When to Use:

- Condition-based loops
- Unknown number of iterations
- Login attempts, retry mechanisms

4. do-while Loop

When the code must run at least once, no matter what.

Syntax:

```
do
{
    Console.WriteLine("Enter password:");
}
while(!IsValidPassword());
```

Codewithindu

Real-World Example:

- ATM PIN input (must show prompt first)
- Display menu once and repeat until exit
- Run validation checks at least once

When to Use:

- User input
- Form validation
- Must execute block at least… ONE TIME

5. Nested Loops

(Loop inside another loop)

```
for(int i = 1; i <= 5; i++)
{
```

```
    for(int j = 1; j <= 5; j++)
    {
        Console.Write(j + " ");
    }
}
```

Real-World Example:

- Generating tables (2D data)
- Matrix operations
- Attendance sheet (students × days)

Q29. How to implement Exception Handling in C#?

Ans-**Exception Handling** in C# is a mechanism to handle runtime errors gracefully, without crashing the application.
It ensures the application continues running or fails safely with a proper message.

C# uses **try, catch, finally, and throw** keywords.

Keywords Used in Exception Handling

try

Block of code where the error may occur.

catch

Handles the exception and provides alternative logic.

finally

Runs **always** (success or error).
Used for **cleanup** (closing connections, files).

throw

Used to manually raise exceptions.

Basic Syntax

```
try
{
    // Code that may throw exception
}
catch(Exception ex)
{
    // Handle exception
}
finally
{
    // Cleanup code (optional)
}
```

Codewithindu

30. What is a Constructor?

Ans- A **constructor** in C# is a special method of a class that is automatically called **when an object is created**.
Its main purpose is to **initialize the object**, assign default values, and prepare the object for use.

Key Characteristics of a Constructor

- Has **same name as the class**
- **No return type** (not even void)
- Called **automatically**
- Used to **initialize variables** or resources
- Can be **overloaded**

Example-

```
public class Employee
{
```

```
    public string Name;


    public Employee()  // Constructor
    {
        Name = "Unknown";
    }
}
```

Employee emp = new Employee(); // Constructor is called automatically

Q31. What are the types of constructors in C#?

In C#, a **constructor** is a special method that is automatically called when an object is created.
C# provides several types of constructors, each serving a unique purpose.

Types of Constructors in C#

1. Default Constructor

A constructor with **no parameters**.

Purpose:

- Initializes default values.
- Runs automatically when no arguments are passed.

Example:

```
public class Student
{
    public Student()
    {
        Console.WriteLine("Default Constructor Called");
```

```
    }
}
```

2. Parameterized Constructor

A constructor that **accepts parameters** to initialize data.

Purpose:

- To pass values at the time of object creation.

Example:

```
public class Student
{
    public string Name;

    public Student(string name)
    {
        Name = name;
    }}
```

3. Copy Constructor

A constructor that copies data from **another object** of the same class.

Purpose:

- Creates a **duplicate object** with the same values.

Example:

```
public class Student
{
    public string Name;
```

Codewithindu

```
    public Student(string name)
    {
        Name = name;
    }

    public Student(Student s)
    {
        Name = s.Name;
    }
}
```

4. Static Constructor

A constructor that initializes **static members** of the class.

Key Points:

- Automatically called **once**.
- No access modifiers.
- Cannot take parameters.

Example:

```
public class LogManager
{
    static LogManager()
    {
        Console.WriteLine("Static Constructor Called");
    }
```

```
}
```

5. Private Constructor

A constructor marked **private**.

Purpose:

- Prevents object creation from outside the class.
- Used in **Singleton Pattern**.
- Used in **utility classes**.

Example:

```
public class Singleton
{
   private static Singleton _instance;

   private Singleton() { }

   public static Singleton GetInstance()
   {
      if (_instance == null)
         _instance = new Singleton();

      return _instance;
   }
}
```

6. Constructor Overloading

Multiple constructors in the same class with **different parameters**.

Purpose:

- Flexibility in object creation.
- Initializes object differently based on parameters.

Example:

```
public class Employee
{
    public Employee() { }
    public Employee(string name) { }
    public Employee(string name, int age) { }
}
```

7. Chained Constructor (this())

One constructor **calls another** using this().

Purpose:

- Reuse initialization logic.
- Avoid duplicate code.

Example:

```
public class Person
{
    public Person() : this("Unknown") { }


    public Person(string name)
    {
        Console.WriteLine(name);
    }
```

codewithindu

```
}
```

Real-world Use Cases

Default Constructor

Used when you want default behavior
e.g., creating an empty cart in an e-commerce app.

Parameterized Constructor

Used when values come from user input or DB
e.g., Student("Payal", 22).

Copy Constructor

Used to duplicate objects
e.g., cloning an order before modifying it.

Static Constructor

Used to initialize configuration
e.g., reading connection string once.

Private Constructor

Used in **Singleton** or **Helper classes**
e.g., Math class cannot be instantiated.

Overloaded Constructors

Used when there are multiple ways to create objects
e.g., product creation with or without discount.

Q32. What is the difference between “out” and “ref” parameters in C#?

Ans-

Feature	**ref**	**out**
Initialization required before passing?	✓ Yes	✗ No
Must assign value inside method?	✗ Not compulsory	✓ Yes, must assign

Feature	ref	out
Purpose	Pass existing value to modify	Return multiple values from a method
Common Use	Update an existing variable	Methods that need to return more than one value

Q33-What are Delegates in C#?

Ans-A **Delegate** in C# is a **type-safe function pointer** that holds the reference of a method.
It allows methods to be **passed as parameters**, stored in variables, and executed dynamically.

Type-safe → return type + parameter type must match
Used for callback methods, event handling, and LINQ

Simple

Think of a **delegate** as a **remote control button**. When you press the button, it calls a method sitting somewhere. You can attach/replace/change the method anytime.

Delegate = A way to call a method without knowing its name at compile time

Q34-What are the differences between Events and Delegates?

Ans-

Feature	Delegate	Event
What it is?	A type-safe function pointer	A wrapper over delegate that provides restricted access
Who can invoke?	Anyone who has the delegate object	Only the class that declares the event
Purpose	Holds method reference	Implements publisher–subscriber model
Usage	Call methods dynamically	Notify subscribers when something happens
Access Control	Less secure	More secure (prevents misuse)

Delegate

A **delegate** is a type-safe function pointer that stores the reference of one or more methods.

Event

An **event** is a special delegate that follows the **publisher–subscriber pattern**, used to notify other classes when something happens.

Events use delegates internally but add safety and restrictions

Simple Explanation

Think of:

- **Delegate → Phone number**
  Anyone can call it.
- **Event → Doorbell**
  Only the person **inside the house (publisher)** can ring it. Outsiders **cannot trigger** it.

Q35. What is the purpose of the "using" keyword in C#?

Ans-The **using keyword** in C# has **two different purposes**, and both are important for interviews.

1: Import Namespaces

The using keyword is used at the top of your file to **include namespaces**, so you can use their classes without writing the full path.

Example

```
using System;

using System.Collections.Generic;

Without using keyword:

System.Console.WriteLine("Hello");

System.Collections.Generic.List<int> numbers = new System.Collections.Generic.List<int>();
```

With using:

```
Console.WriteLine("Hello");

List<int> numbers = new List<int>();
```

Real-world example

When building an ASP.NET or console app, you include namespaces like:

```
using System.IO;

using System.Linq;

using System.Net.Http;
```

2: Automatic Resource Disposal

using is used as a **using block** to ensure that objects like:

- File handlers
- Database connections
- Streams
- Network connections

are **automatically closed and disposed** even if an exception occurs.

This prevents **memory leaks**
Ensures **clean-up**
Works for classes that implement **IDisposable**

Example of Using Block

```
using (StreamReader sr = new StreamReader("data.txt"))

{

    string content = sr.ReadToEnd();

    Console.WriteLine(content);

}
```

Q36. What is “Static” class? When to use it?

Ans-A **static class** in C# is a class that:

- **Cannot be instantiated** (you cannot create objects of it)
- Contains **only static members** (static methods, static variables, static properties)
- Is loaded once in memory for the entire application

It is declared using the static keyword:

```
public static class MathHelper
{
    public static int Add(int a, int b) => a + b;
}
```

OOPS&C#-LINQ

Q37. What is LINQ? When to use LINQ in real applications?

Ans-**LINQ (Language Integrated Query)** is a powerful feature in C# that allows you to write **queries** (like SQL) directly in C# to retrieve and filter data from:

- Collections (List, Array)
- Databases (Entity Framework)
- XML files
- Objects
- Remote services

LINQ provides a **consistent**, **type-safe**, and **readable** way to query data. LINQ is like writing **SQL inside C#** to filter, sort, group, and select data.

Example:

```
var result = students.Where(s => s.Age > 18).ToList();
```

This feels like SQL:

```
SELECT * FROM Students WHERE Age > 18;
```

Q38-What are the advantages & disadvantages of LINQ?

Ans. Advantages of LINQ

1. Cleaner and Shorter Code

LINQ reduces multiple lines of loops, conditions, and temporary collections into a **single readable line**.

Example:

```
var highSalary = employees.Where(e => e.Salary > 50000).ToList();
```

2. Readability and Maintainability

LINQ code looks like natural English/SQL → easy to read and understand.

3. Type Safety (Compile-Time Checking)

LINQ expressions are checked at compile time, reducing runtime errors.

SQL is string-based; LINQ is **object-based**.

4. Works with Any Data Source

LINQ can query:

- Lists
- Arrays
- Databases (EF)
- XML
- JSON
- Collections

5. Eliminates Manual Loops

No need for foreach, temporary lists, nested loops, etc.

6. Supports Powerful Operations

LINQ provides:

- Filtering (Where)
- Sorting (OrderBy)

- Grouping (GroupBy)
- Joining (Join)
- Aggregation (Count, Sum, Average)

These are difficult with loops but easy with LINQ.

B. Disadvantages of LINQ

1. Performance Overhead in Some Cases

LINQ can be slower than traditional loops in:

- Very large data
- Complex queries
- Tight loops

Because LINQ uses extra layers (enumerators, delegates).

2. Hard to Debug Complex LINQ Queries

Nested LINQ chains can become complex:

.Where(...)

.Select(...)

.GroupBy(...)

Debugging becomes difficult.

3. Deferred Execution Can Cause Unexpected Results

If data changes before execution → output changes unexpectedly.

4. Less Control Over SQL (When Using LINQ-to-SQL/EF)

LINQ may generate:

- Inefficient SQL
- Unwanted joins
- Extra columns

This affects SQL performance.

Codewithindu

5. Learning Curve

Developers must understand:

- Lambda expressions
- Delegates
- Expression trees
- Deferred execution

Beginners may struggle.

6. Not Ideal for Highly Complex Queries

Very complex joins, window functions, stored procedures → SQL is better.

Q39: What is the difference between First() and FirstOrDefault() in LINQ?

Ans-First()

- Returns **the first element** of a sequence.
- If the sequence is **empty**, it throws an **InvalidOperationException**.
- Use when you are **sure** the collection contains at least one element.

Example:

```
var result = employees.First(e => e.Salary > 50000);
```

If no employee satisfies the condition → ✕ Exception thrown.

FirstOrDefault()

- Returns **the first element**, or **default value** if the sequence is empty.
- Default value:
  - null for reference types
  - 0, false, etc. for value types.
- Use when the collection **might be empty** and you want to **avoid exceptions**.

Example:

```
var result = employees.FirstOrDefault(e => e.Salary > 50000);
```

If no match found → it returns **null**, not an exception.

Key Differences

Feature	First()	FirstOrDefault()
Returns	First matching element	First matching element or default value
When no element found	✕ Throws exception	✓ Returns default (null)
Safety	Less safe	Safe (No exception)
Use case	When data **must exist**	When data **may not exist**

.NET Framework- Basics

Q40. What are the important components of .NET framework?

Ans-The **.NET Framework** is a software development platform created by Microsoft. It provides an environment where you can build, run, and manage applications.
Its important components are:

1. Common Language Runtime (CLR)

- The **heart of the .NET Framework**.
- Manages the **execution** of .NET programs.
- Handles:
  - Memory management
  - Garbage collection
  - Exception handling
  - Thread management
  - Security
- Converts **IL (Intermediate Language)** code into **machine code** using JIT.

Interview
CLR = Runtime engine + Services.

2. .NET Class Library / Base Class Library (BCL)

- A huge collection of **reusable classes, interfaces, and types**.
- Provides built-in functionality such as:
  - File I/O

- String manipulation
- Collections
- Networking
- Security
- Data access

- Reduces development time since we don't need to write everything from scratch.

3. Common Type System (CTS)

- Defines **how data types are declared and used** in .NET.
- Ensures all .NET languages (C#, VB.NET, F#) share the **same data type rules**.
- Supports value types & reference types.

Purpose:
Different languages can communicate easily.

4. Common Language Specification (CLS)

- A set of **rules and guidelines** that all .NET languages must follow.
- Ensures **interoperability** between languages.
  Example:
  If you write a class in C#, VB.NET code can also use it.

5. Assemblies & Metadata

- **Assemblies** are the building blocks of .NET applications (.dll, .exe).
- Contain:
  - IL code
  - Metadata
  - Manifest
- Metadata stores information about:
  - Types
  - Methods
  - Properties
  - References

6. ASP.NET (for Web Applications)

- Framework for building **websites, APIs, and services**.
- Supports:

- MVC
- WebForms
- Web API
- Razor Pages

7. ADO.NET (for Database Access)

- Provides APIs for interacting with databases.
- Supports SQL Server, Oracle, MySQL, and more.
- Components:
  - Connection
  - Command
  - DataReader
  - DataSet

8. Windows Forms / WPF (for Desktop Applications)

- **WinForms** → Simple GUI applications
- **WPF** → Modern UI with XAML, animations, data-binding

9. LINQ (Language Integrated Query)

- Query data in a **consistent** way (SQL, XML, Objects).
- Improves readability and reduces code complexity.

Q41. What is Serialization and Deserialization? What are the types of Serialization?

Ans-Serialization is the process of **converting an object into a format** that can be:

- Stored (in file, database, cache)
- Transmitted (over network, API)
- Reconstructed later

Common formats: **JSON, XML, Binary, SOAP**.

Simple Definition: *Serialization = Object → Data Format*

Deserialization is the reverse process.
It **converts stored/transmitted data back into an object**.

Simple *Deserialization = Data Format → Object*

Real-World Example

Imagine you want to send a **parcel** to your friend:

- You **pack items into a box** → this is **serialization**
- The courier delivers it
- Your friend **opens the box and takes out items** → this is **deserialization**

Similarly, C# "packs" objects into JSON, XML, or Binary before sending.

Serialization is Used

- To save object state (session storage, caching)
- To send objects over network (API communication)
- To store data easily in files
- To transfer data between different systems or languages

Types of Serialization in .NET

1. Binary Serialization (Fastest)

- Stores **complete object state** including private fields.
- Very fast but **not human readable**.

```
BinaryFormatter formatter = new BinaryFormatter();

formatter.Serialize(stream, obj);
```

2. XML Serialization (Readable)

- Only serializes **public properties**.
- Human-readable.

```
XmlSerializer serializer = new XmlSerializer(typeof(Student));

serializer.Serialize(stream, student);
```

3. JSON Serialization (Most Popular)

- Light-weight, widely used in web APIs.

```
string json = JsonConvert.SerializeObject(obj);
```

4. SOAP Serialization

- XML-based serialization for SOAP services (legacy).
- Rarely used today.
- **5. Custom Serialization**

- When you want to **manually control** what to serialize.
- Implement ISerializable interface.

Q42. What is the difference between synchronous and asynchronous programming?

Ans-**1. Synchronous Programming**

In **synchronous programming**, tasks run **one after another**, in a **blocking** manner.

The next task will only start after the previous task completes.

Real-World Example (Simple):

Standing in a line at a ticket counter — each person waits until the previous person is done.

Example in C#:

```
var data = GetData(); // waits until this is complete

Console.WriteLine("Next Task");
```

2. Asynchronous Programming

In **asynchronous programming**, tasks run **without blocking**. The program continues executing while the task runs in the background.

Allows multiple tasks to run concurrently.

Real-World Example:

Ordering food online —
you place the order → continue doing your work → food arrives later.

Example in C#:

```
var data = await GetDataAsync();

Console.WriteLine("Next Task"); // runs while data is loading
```

Q43. What is the role of async and await in C#?

Ans-**1. async in C#**

async is a **modifier** used in a method declaration to enable **asynchronous programming**.

It tells the compiler that this method **may contain** asynchronous operations. It allows the use of **await** inside the method. Returns a **Task** or **Task<T>** (or void for event handlers).

Example:

```
public async Task LoadDataAsync()

{

    // async code here

}
```

async itself does **not** make the method run asynchronously — it simply **enables** the use of await.

2. await in C#

await is an **operator** used to **asynchronously wait** for a task to complete, *without blocking the thread*.

It pauses the method **only at that line**, But **releases the thread** to continue other work. When the awaited task finishes, execution resumes automatically.

Example:

```
string data = await GetDataFromApiAsync();
```

The UI does **not freeze**, and the thread is free to serve other requests.

How async & await Work Together (Simple Real-World Example)

Example: Ordering Pizza (Async Method)

- You order pizza → The restaurant prepares it → Meanwhile, you continue working.

Code Version:

```
public async Task OrderPizzaAsync()
{
    Console.WriteLine("Order placed...");
    await Task.Delay(5000); // simulating pizza making time
    Console.WriteLine("Pizza Delivered!");
}
```

- async → allows the method to run with asynchronous behavior
- await → pauses at Task.Delay(5000) but **does not block the thread**

Entity Framework

Q44. What is Entity Framework (EF)?

Ans-**Entity Framework (EF)** is an **Object-Relational Mapper (ORM)** for .NET applications.
It allows developers to interact with a database **using C# classes** instead of writing SQL queries manually.

EF converts C# code to SQL and SQL results back to C# objects.

EF Used

- To reduce the need for **manual SQL queries**
- To speed up development
- To maintain **clean, object-oriented code**
- To avoid database-specific complexity
- To handle CRUD (Create, Read, Update, Delete) automatically

Real-World Example (Easy Explanation)

Imagine you have a **notebook** where you store student details manually. Now you switch to a **software** where you enter details and the system stores them automatically.

You interact with forms (objects)
The software handles writing to the database (SQL)

This is what **EF does** in software development.

Example in C#: EF Converts C# → SQL

C# Code:

```
var students = dbContext.Students.ToList();
```

EF Automatically Generates SQL:

```
SELECT * FROM Students;
```

You don't write SQL — EF writes it for you.

Types of Entity Framework Approaches

1. Code First

- Write **C# classes first**, EF creates database. Used in modern apps.

2. Database First

- Database already exists → EF generates classes. Best for legacy databases.

3. Model First-Create a model diagram → EF generates DB + classes. Rare today.

Q45. What is meant by DBContext and DBSet?

Ans-**Q45. What is meant by DbContext and DbSet in Entity Framework?**

Ans- **DbContext**

DbContext is the **main class** in Entity Framework responsible for:

- Connecting to the **database**
- Managing **database operations**
- Tracking **changes in objects**
- Saving data to the DB (SaveChanges())
- Providing access to database tables via **DbSet**

In simple words:
DbContext = Bridge between your C# application and the database.

Real-World Example

Think of **DbContext** as a **manager** in a store.

- The manager knows how to communicate with the warehouse (DB)
- You ask the manager to get items, update items, or remove items
- The manager does all heavy tasks for you

Thus, DbContext manages all database-related work.

Example Code

```
public class SchoolContext : DbContext
{
    public DbSet<Student> Students { get; set; }
    public DbSet<Course> Courses { get; set; }
}
```

Here,

- **SchoolContext** = DbContext
- **Students**, **Courses** = Tables of database

DbSet

DbSet represents a **table** in the database.

Each **DbSet<T>** represents a collection of entities. Allows you to perform **CRUD operations**.

DbSet = Database Table

Example:

```
public DbSet<Student> Students { get; set; }
```

This means:

- Students is a table in the database
- Each row = a Student object
- You can do:

```
context.Students.Add(student);      // INSERT

context.Students.ToList();          // SELECT

context.Students.Update(student);  // UPDATE

context.Students.Remove(student);  // DELETE
```

Relationship Between DbContext and DbSet

DbContext

|

|-- DbSet<Student> → Students Table

|-- DbSet<Teacher> → Teachers Table

|-- DbSet<Course> → Courses Table

DbContext contains DbSets
DbSet represents tables

Q46. How Entity Framework Works? ORM How to Set Up Entity Framework?

Ans**1. What Is Entity Framework Actually Doing**

Entity Framework (EF) is an **ORM** that:

1. Maps **C# classes → Database Tables**
2. Converts **LINQ queries → SQL queries**
3. Tracks changes in objects
4. Saves changes to database

5. Loads data as C# objects (not raw SQL)

EF acts as a **middle layer** between your application and database.

2. How Entity Framework Works

C# Classes → EF ORM → SQL Database

(Models) (Mapping + SQL)

Flow:

You write C# code → EF converts to SQL → DB executes → EF returns objects

3. Step-by-Step: How to Set Up EF (EF Core)

Below is the **standard and most correct way**, used in real .NET Core projects.

Step 1: Install EF Core Packages

Run these commands in the Package Manager Console:

Install-Package Microsoft.EntityFrameworkCore

Install-Package Microsoft.EntityFrameworkCore.SqlServer

Install-Package Microsoft.EntityFrameworkCore.Tools

Step 2: Create Your Model Classes (Entities)

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Age { get; set; }
}
```

These represent **database tables**.

Step 3: Create DbContext Class

```
public class SchoolContext : DbContext

{

   public SchoolContext(DbContextOptions<SchoolContext> options) : base(options) { }


   public DbSet<Student> Students { get; set; }

}
```

DbContext = Manages database

DbSet = Table (Students)

Step 4: Add Connection String (appsettings.json)

```
"ConnectionStrings": {

 "SchoolDb": "Server=.;Database=SchoolDB;Trusted_Connection=True;"

}
```

Step 5: Register DbContext in Program.cs / Startup.cs

```
builder.Services.AddDbContext<SchoolContext>(options =>


options.UseSqlServer(builder.Configuration.GetConnectionString("SchoolDb")))
;
```

This tells .NET to use EF with SQL Server.

Step 6: Apply EF Migrations (Code First)

Create migration:

```
Add-Migration InitialCreate
```

Update database (create tables):

Update-Database

EF automatically creates tables in the database.

Step 7: Use EF in Application (CRUD Operations)

Insert:

```
var student = new Student { Name = "Amit", Age = 20 };

context.Students.Add(student);

context.SaveChanges();
```

Read:

```
var data = context.Students.ToList();
```

Update:

```
student.Name = "Amit Kumar";

context.SaveChanges();
```

Delete:

```
context.Students.Remove(student);

context.SaveChanges();
```

4. Real-World Example — EF in a Web API

```
[HttpGet]

public async Task<IActionResult> GetStudents()

{

    var students = await _context.Students.ToListAsync();

    return Ok(students);

}
```

EF automatically generates SQL
Executes it
Returns a list of Student objects

5. How EF Executes a Query (Behind the Scenes)

Your Code:

```
var list = context.Students.ToList();
```

EF Converts to SQL:

```
SELECT * FROM Students;
```

Database returns rows

EF converts rows → C# Objects
C# Objects returned to your code

Entity Framework works as an ORM that maps C# classes to database tables. It converts LINQ queries into SQL, executes them, and returns results as objects. To set up EF, you install EF packages, create models, create DbContext, add connection string, register DbContext, apply migrations, and then perform CRUD using DbSet.

Q47. What is the difference between LINQ to SQL and Entity Framework?

Ans-

Feature	**LINQ to SQL**	**Entity Framework (EF)**
Release	Introduced with .NET 3.5	Introduced later, more advanced
Database Support	Supports **only SQL Server**	Supports **multiple databases**: SQL Server, Oracle, MySQL, PostgreSQL, SQLite, etc.
Architecture	Simple ORM (Object Relational Mapper)	Full-fledged ORM with rich features
Mapping Support	Maps **one database table = one class**	Supports **complex mapping** (TPT, TPH, TPC, complex types)
Features	Basic CRUD	Advanced features: migrations, lazy loading, eager loading, unit of work, LINQ, relationships
Relationships	Limited relationship support	Full support (one-to-one, one-to-many, many-to-many)

Feature	LINQ to SQL	Entity Framework (EF)
Modeling	Uses DBML designer	Supports Code First, Database First, and Model First
Use Cases	Small, simple applications	Medium to large enterprise applications

Simple Explanation

LINQ to SQL → Small tool, works only with SQL Server

- It directly maps SQL Server tables to .NET classes.
- Good only for simple projects.
- Very limited features.

Entity Framework → Big tool, supports all major databases

- More powerful and flexible.
- Supports advanced mappings and migrations.
- Used in real-world, enterprise applications.

Example

LINQ to SQL:

```
DataContext db = new DataContext(connectionString);

var students = db.GetTable<Student>().ToList();
```

Entity Framework:

```
using (var context = new SchoolDbContext())

{

    var students = context.Students.ToList();

}
```

Q48.What are the different types of development approaches used with EF?

Ans-Entity Framework provides **three main development approaches**:

Code First Approach

You write C# classes first, and the database is created automatically from those classes.

Best for:

- New projects
- Fully controlled model
- Agile development

Example:

```
public class Student

{

   public int Id { get; set; }

   public string Name { get; set; }

}
```

EF will generate the database and tables using **migrations**.

Advantages:

- Total control on C# model
- Easy to maintain
- Supports migrations

2️⃣ Database First Approach

You create the database first, and EF will generate C# classes (Entities) from the existing database.

Best for:

- When the database already exists
- Enterprise-level DB designed by a DB team

Workflow:

- Create DB → Create tables → Use EF Designer / Scaffold-DbContext

Advantages:

- Perfect for legacy databases
- Auto-generate models and relationships

3️⃣ Model First Approach

You design the Entity Data Model visually using EDMX, and then EF generates both the classes and database.

Best for:

- When you want a visual diagram
- Designing conceptual model first

Workflow:

- Create .edmx diagram → EF creates DB + Classes

Advantages:

- Visual representation
- Good for complex relationships

Note:

Model First is used mostly in **old EF (not EF Core)**.

Q49. What is ORM? What are the different types of ORM?

Ans-**ORM (Object–Relational Mapping)** is a technique that allows developers to interact with a database using **C# objects instead of SQL queries**.

It converts **C# classes** → **database tables**
It converts **objects** → **rows**
It reduces the need to write complex SQL manually
It improves development speed and removes repetitive code

ORM is a bridge between object-oriented code and relational databases.

Why do we need ORM

Without ORM → You have to write SQL manually:

SELECT * FROM Students WHERE Id = 1;

With ORM → You write:

var student = db.Students.FirstOrDefault(x => x.Id == 1);

ORM automatically converts this LINQ code into SQL.

Main Benefits of ORM

- Faster development
- Less SQL to write
- Automatic mapping
- Supports migrations
- Better maintainability
- Built-in CRUD operations

Different Types of ORM

There are many ORMs used across programming languages. But in **.NET**, the most commonly used ORMs are:

1️⃣ Entity Framework (EF / EF Core) – (Most Popular in .NET)

Microsoft's ORM
Uses LINQ
Supports Code First / DB First
Automatic migrations

Real-world use → **Almost every .NET + SQL Server project uses EF Core 2️⃣**

NHibernate

Powerful ORM
Inspired by Java's Hibernate
Older but still used in large projects
Has advanced caching and mapping

Used in → Banking, finance, legacy enterprise apps

3️⃣ Dapper (Micro ORM)

Lightweight ORM
Very fast (almost raw SQL speed)

Does not generate SQL automatically
You write SQL, it maps results to C# objects

Example:

```
connection.Query<Employee>("SELECT * FROM Employees");
```

Used for → High-performance applications

4️⃣ LINQ to SQL (Old)

Older Microsoft ORM
Only supports SQL Server
Replaced by Entity Framework

Now **not preferred** in new applications.

WebAPI- Basics

Q50: What is Web API? What is the purpose of Web API?

Ans-**Web API (Application Programming Interface)** in .NET is a framework used to build **HTTP-based services** that can be consumed by any client:
Angular
React
Mobile Apps
Desktop Apps
IoT Devices
Other backend systems

Simple Definition:

Web API is used to send and receive data over HTTP using JSON. It is used to build RESTful services.

Example:

A Web API method to fetch students:

```
[HttpGet]
public IEnumerable<Student> GetStudents()
```

```
{
    return _context.Students.ToList();
}
```

This API can be called by:

- Android app
- Angular app
- Another .NET service

Purpose of Web API

Web API is mainly used to **communicate between Client and Server**.

1. To Build RESTful Services

REST means:

- GET → Fetch data
- POST → Insert data
- PUT → Update data
- DELETE → Delete data

Web API makes it easy to build these operations.

2. To Allow Different Applications to Communicate

A Web API exposes data so that any external system can consume it.

Example:
A mobile app asking the server for user details:

Mobile App → (API Call) → Web API → (JSON Response)

3. Platform Independent Communication

Web API uses **JSON**, which works everywhere.

Even Java, Python, PHP can consume .NET Web API.

4. Lightweight & Fast

Unlike WCF, Web API is:

- Lightweight
- Uses HTTP directly
- Easy to host
- Faster for CRUD operations

5. Used for Modern Web Applications

Angular / React frontend → .NET Web API backend.

This is the **most common architecture** today.

Q51. What are HTTP Verbs or HTTP Methods?

Ans-HTTP Verbs (also called **HTTP Methods**) are the **actions** that a client (like Angular, React, Postman, Mobile App) sends to the server (Web API) to specify **what operation** it wants to perform.

They define **what type of request** is being made.

HTTP Method	Purpose	CRUD Mapping	Example
GET	Fetch/read data	Read	Get list of students
POST	Insert/create new data	Create	Add a new student
PUT	Update existing data (full update)	Update	Update all details of a student
PATCH	Partial update	Update	Update only email of a student
DELETE	Remove data	Delete	Delete a student

1. GET – Read Data

Used to **retrieve** data from server.

Example in C# Web API:

```
[HttpGet]
public IEnumerable<Student> GetStudents() { }
```

Client example:

GET https://api.myapp.com/students

2. POST – Create New Data

Used to **insert** new records.

```
[HttpPost]
public IActionResult AddStudent(Student student) { }
```

Client example:

```
POST https://api.myapp.com/students
Body:
{
  "name":"Payal",
  "age":22
}
```

Codewithindu

3. PUT – Update Entire Record

Used for **full update** (replace old data with new data).

```
[HttpPut("{id}")]
public IActionResult UpdateStudent(int id, Student model) { }
```

4. PATCH – Partial Update

Allows you to update **only specific fields**.

Example:

```
[HttpPatch("{id}")]
public IActionResult UpdateEmail(int id, JsonPatchDocument<Student>
patchDoc) { }
```

PATCH request updates only **email**, not whole student object.

5. DELETE – Remove Data

Used to delete a record.

```
[HttpDelete("{id}")]

public IActionResult DeleteStudent(int id) { }
```

Q52.What is the Difference Between REST API and Web API?

Ans-**1. Web API (ASP.NET Web API)**

Web API is a **Microsoft framework** used to build HTTP-based services in .NET applications.

Web API **can follow REST principles**,
but it is **not limited to REST only**
(It can return data in XML, JSON, BSON, etc.)

2. REST API (Representational State Transfer API)

REST API is an **architectural style** or **set of rules** used to design network-based services.

✓ REST is **not a technology**
✓ REST is **not a Microsoft concept**
✓ It is a **standard for designing APIs**

REST APIs follow rules like:

- Client–Server
- Stateless
- Cacheable
- Uniform Interface
- Use of HTTP verbs (GET, POST, PUT, DELETE)

Simple Explanation

Feature	**Web API**	**REST API**
Definition	A framework in .NET to build APIs	A design style for building APIs

Feature	Web API	REST API
Owner	Microsoft	No owner (standard architecture)
Format Support	JSON, XML, BSON, Form-data, etc.	Mostly JSON (but can support others)
Protocol	Uses HTTP/HTTPS	Uses HTTP/HTTPS
Can be REST?	Yes	REST API must follow REST rules
Scope	Bigger (supports non-REST too)	Smaller (only REST-style)
Platform	.NET-only	Any platform (Java, Python, Node.js, etc.)

Easy Real-World Example

Web API Example (ASP.NET):

```
[HttpGet]
public IEnumerable<Student> GetStudents() { }
```

REST API Rule Example:

Calling:

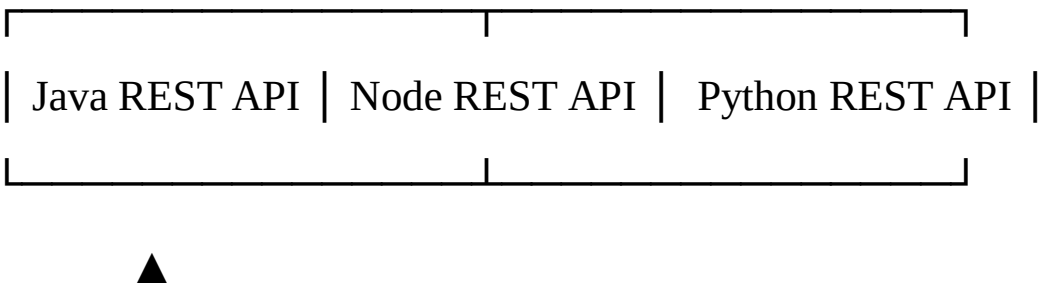
GET /students

must return all students — stateless, structured, and predictable.

REST cares **how** you design the service.
Web API cares **how** you build it in .NET.

Diagram

REST API (Architecture)



▲

Web API (.NET framework to build APIs)

Q53. What is the difference between Web API and MVC Controller?

Ans-**1. ASP.NET MVC Controller**

Used to **build web applications** that return **HTML views** for browsers.

Outputs:

- HTML pages
- Razor views
- Partial views

Used For:

- Websites (UI + HTML)
- Server-side rendered apps

Example:

```
public class HomeController : Controller
{
    public IActionResult Index()
    {
        return View(); // returns HTML View
    }
}
```

2. ASP.NET Web API Controller

Used to **build RESTful services** that return **data (JSON/XML)** to clients.

Outputs:

- JSON (mostly)
- XML

- HTTP status codes

Used For:

- Mobile apps
- Angular/React apps
- Third-party integrations
- Microservices

Example:

```
[ApiController]

[Route("api/[controller]")]

public class StudentsController : ControllerBase

{

   [HttpGet]

   public IEnumerable<Student> GetStudents()

   {

      return _context.Students.ToList(); // returns JSON

   }

}
```

Feature	MVC Controller	Web API Controller
Purpose	Serve HTML UI	Serve JSON data
Base Class	Controller	ControllerBase
Return Type	ViewResult, IActionResult, HTML	JSON, XML, IActionResult
Used For	Web applications	RESTful APIs
Request/Response	Browser-based	Any client (Mobile, Angular, Postman)
Routing	Conventional Routing	Attribute Routing

Feature	MVC Controller	Web API Controller
Content Negotiation	Limited	Strong (JSON/XML)

WebAPI- Authentication & JWT

Q54. What are the types of authentication techniques in Web API?

Ans-*Authentication is the process of verifying the identity of a user or client trying to access an API.*
In Web API, authentication ensures that **only authorized clients** can access resources.

1. Basic Authentication

- **Definition:** Sends username and password encoded in Base64 in the HTTP header.
- **Usage:** Simple internal APIs, testing with Postman.
- **Example:**

Authorization: Basic dXNlcjp1c2VyMTIz

- **Note:** Must use HTTPS; otherwise, credentials are insecure.

2. Token-Based Authentication

- **Definition:** Server generates a token on login; client sends token with each request.
- **Usage:** Modern web apps, SPAs, mobile apps.
- **Example Header:**

Authorization: Bearer <token>

3. JWT (JSON Web Token)

- **Definition:** Self-contained, signed token containing user info and expiry.
- **Usage:** Single Page Applications, mobile apps, e-commerce.
- **Example:**

Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6...

- **Advantage:** Stateless, no server storage needed.

4. OAuth 2.0

- **Definition:** Industry-standard framework for **secure delegated access** using access and refresh tokens.
- **Usage:** Social logins (Google, Facebook, GitHub), enterprise apps.
- **Example:**
  Client receives access_token and refresh_token after login via OAuth provider.

5. API Key Authentication

- **Definition:** Client sends a unique API key in headers or query string.
- **Usage:** External APIs (Google Maps, Payment gateways, Weather APIs).
- **Example:**

x-api-key: 123456abcdef

- **Note:** Simple but less secure.

Codewithindu

6. Windows Authentication

- **Definition:** Uses the client's Windows credentials (Active Directory).
- **Usage:** Intranet applications, internal dashboards.

7. Certificate-Based Authentication

- **Definition:** Client must provide a valid **digital certificate**.
- **Usage:** Banking, government APIs, high-security applications.

8. Digest Authentication

- **Definition:** Similar to Basic Auth, but credentials are **hashed** before sending.
- **Usage:** Legacy systems.

Key Points for Interviews

- **Modern Web APIs** → Use **JWT or OAuth 2.0**.
- **Internal apps** → Windows Authentication or Basic Auth.
- **External APIs** → API Key.
- **High-security apps** → Certificates.

“Web API supports multiple authentication techniques: Basic Authentication, Token-Based, JWT, OAuth 2.0, API Key, Windows Authentication, Certificate-Based, and Digest.
Basic sends username/password; Token and JWT s”

Q55: What is Basic Authentication in Web API?

Ans- **Basic Authentication** is a simple authentication technique in which the client sends the **username and password** encoded in **Base64** in the HTTP request header.
The server validates the credentials and allows access if correct.

Key point: The credentials are sent **with every request**, so it must be used with **HTTPS** to ensure security.

How it Works (Step-by-Step)

1. Client sends HTTP request with **Authorization header**:

```
Authorization: Basic <Base64Encoded(username:password)>
```

2. Server decodes the Base64 string to get **username and password**.
3. Server checks credentials against the database or user store.
4. If valid → Server responds with requested data (200 OK)
   If invalid → Server responds with 401 Unauthorized

Example in Web API (C#)

```
[Authorize]

[HttpGet]

public IActionResult GetStudents()

{

   return Ok(_context.Students.ToList());

}
```

Client Request (Header Example):

```
GET /api/students
```

Authorization: Basic dXNlcjpwYXNzd29yZA==

- Here dXNlcjpwYXNzd29yZA== is Base64 for user:password

Real-World Use

- Internal company APIs
- Testing APIs with Postman
- Quick prototypes or internal tools

Not recommended for public-facing APIs without HTTPS because credentials can be intercepted.

Advantages

- Simple to implement
- Supported by almost all HTTP clients
- No cookies or session needed

Disadvantages

- Credentials are sent with every request
- Not secure without HTTPS
- Harder to scale for large applications

Codewithindu

"Basic Authentication in Web API is a method where the client sends username and password encoded in Base64 in the HTTP header. The server decodes and validates them. It is simple and widely supported, but must be used with HTTPS for security. Commonly used for testing, internal apps, or prototypes."

Q56: What is Token-Based Authentication in Web API?

Ans-**Token-Based Authentication** is a method in which the server generates a **token** (a string of characters) after validating user credentials. The client sends this token with every subsequent API request instead of username and password.

Tokens are **secure, self-contained**, and often have an **expiry time**.

How it Works (Step-by-Step)

1. Client sends **username and password** to the login endpoint.
2. Server verifies credentials.
3. If valid, server generates a **token** (e.g., JWT) and sends it to the client.

4. Client stores the token (in memory, local storage, or cookies).
5. For every API request, client includes the token in the header:

Authorization: Bearer <token>

6. Server validates the token before granting access to resources.

Example in ASP.NET Core Web API

Login Endpoint (Generate Token):

```
[HttpPost("login")]
public IActionResult Login([FromBody] UserLogin login)
{
    if (login.Username == "admin" && login.Password == "123")
    {
        var token = TokenService.GenerateToken(login.Username);
        return Ok(new { Token = token });
    }
    return Unauthorized();
}
```

Client Request with Token:

```
GET /api/students
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Server validates the token and allows access.

Real-World Use

- **Angular / React SPA apps** → Token stored in local storage.
- **Mobile apps** → Token sent with each API request.
- **Microservices** → Token ensures stateless authentication between services.

Q57: What is JWT Authentication?

Ans-**JWT (JSON Web Token) Authentication** is a type of **token-based authentication** where the server issues a **self-contained token** after validating credentials.

- The token contains **user information**, **claims**, and **expiry time**.
- The token is **digitally signed** using a secret key, ensuring its authenticity.

JWT allows **stateless, secure communication** between client and server.

How JWT Works (Step-by-Step)

1. Client sends **username and password** to the login API.
2. Server validates credentials.
3. Server generates a **JWT token** and sends it to the client.
4. Client stores the token (LocalStorage, SessionStorage, or Cookies).
5. Client sends the JWT in the **Authorization header** for each API request:

   Authorization: Bearer <JWT-Token>

6. Server validates the token signature and expiry before granting access.

JWT Structure

A JWT has **three parts**, separated by dots:

Header.Payload.Signature

- **Header:** Algorithm + token type
- **Payload:** User info + claims + expiry
- **Signature:** Encoded hash to verify authenticity

Example Token:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.

eyJ1c2VybmFtZSI6Ik5pZnR5IiwiZXhwIjoxNzAxMjM0NTY3fQ.

dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk

Example in ASP.NET Core Web API

Generate JWT Token:

Codewithindu

```
var tokenHandler = new JwtSecurityTokenHandler();
var key = Encoding.ASCII.GetBytes("SecretKey12345");
var tokenDescriptor = new SecurityTokenDescriptor
{
    Subject = new ClaimsIdentity(new Claim[]
    {
        new Claim(ClaimTypes.Name, user.Username)
    }),
    Expires = DateTime.UtcNow.AddHours(1),
    SigningCredentials = new SigningCredentials(new SymmetricSecurityKey(key), SecurityAlgorithms.HmacSha256Signature)
};
var token = tokenHandler.CreateToken(tokenDescriptor);
var tokenString = tokenHandler.WriteToken(token);
return Ok(new { Token = tokenString });
```

Client Request with Token:

```
GET /api/students
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Real-World Use

- **Single Page Applications (SPA)**: Angular, React
- **Mobile Apps**: iOS, Android
- **Microservices**: Stateless authentication between services
- **E-commerce Applications**: Securely authenticate users

Q58: What are the parts of a JWT Token?

Ans-A **JWT (JSON Web Token)** is a compact, self-contained token used to **securely transmit information** between client and server.
It consists of **three parts**, separated by dots (.):

Header.Payload.Signature

Header

- **Purpose:** Contains metadata about the token, like the **type** and **algorithm** used for signing.
- **Common Fields:**
- {
- "alg": "HS256",
- "typ": "JWT"
- }
- **Explanation:**
  - alg → Algorithm used to sign the token (e.g., HMAC SHA256)
  - typ → Token type (JWT)

Payload

- **Purpose:** Contains **claims** — the data we want to transmit.
- **Common Claims:**
  - sub → Subject (user id)
  - name → Username
  - iat → Issued at time
  - exp → Expiry time

Example Payload:

```
{
 "username": "Nifty",
 "role": "Admin",
 "exp": 1701234567
}
```

Note: Payload is **not encrypted**; it is only **Base64Url encoded**, so sensitive data should not be stored here.

Signature

- **Purpose:** Ensures the token **has not been tampered with**.
- **How It's Created:**

```
HMACSHA256(
  base64UrlEncode(header) + "." + base64UrlEncode(payload),
  secretKey
)
```

- **Explanation:**
  - Server uses a **secret key** to sign the token
  - Client or server can **verify the token's integrity** using the signature

JWT Token Example

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
.
eyJ1c2VybmFtZSI6Ik5pZnR5Iiwicm9sZSI6IkFkbWluIiwiZXhwIjoxNzAxMjM0NTY3fQ
.
dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

- **Part 1 (Header):** eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
- **Part 2 (Payload):** eyJ1c2VybmFtZSI6Ik5pZnR5Iiwicm9sZSI6IkFkbWluIiwiZXhwIjoxNzAxMjM0NTY3fQ
- **Part 3 (Signature):** dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk

.NET Core- Basics

Q59: What is .NET Core?

Ans-**.NET Core** is a **cross-platform, open-source framework** developed by Microsoft for building **modern applications**.
It allows developers to create:

- **Web applications** (ASP.NET Core)
- **Desktop applications** (with .NET MAUI / Windows Forms / WPF)
- **Microservices**
- **APIs and cloud-based applications**

Unlike the original .NET Framework, **.NET Core runs on Windows, macOS, and Linux**.

Key Features of .NET Core

Feature	Description
Cross-Platform	Works on Windows, Linux, macOS
Open Source	Community-driven, hosted on GitHub
High Performance	Optimized for cloud and microservices
Modular	Use only the libraries you need (NuGet packages)
Side-by-Side Versioning	Multiple .NET Core versions can run on same machine
Unified Development	Supports Web, API, Mobile, Desktop apps

Real-World Examples

- **Netflix, StackOverflow, GoDaddy** → Use .NET Core for APIs and high-performance web services
- **E-commerce platforms** → ASP.NET Core backend for web and mobile apps
- **Microservices & Cloud apps** → Azure services

Q60: What is .NET Standard?

Ans.**.NET Standard** is a **formal specification of APIs** that all .NET platforms must implement.

It acts as a **common library interface** so that **code can run on multiple .NET platforms** (like .NET Core, .NET Framework, Xamarin) without modification.

In simple words:

- .NET Standard defines **what methods and classes should be available**.
- Any library targeting .NET Standard can work on **all compatible .NET platforms**.

Why .NET Standard

- Without .NET Standard, sharing code between .NET Core, .NET Framework, and Xamarin was difficult.
- It solves the **code portability problem**: Write once, run anywhere.

Versioning

.NET Standard Version	Supported Platforms
1.0 – 1.6	Older platforms (limited APIs)
2.0	.NET Core 2.0+, .NET Framework 4.6.1+, Xamarin
2.1	.NET Core 3.0+, Xamarin

Higher .NET Standard version → More APIs available → Fewer platforms support it.

Real-World Example

Imagine you are creating a **library for handling payments**:

```
public class PaymentProcessor
{
    public bool ProcessPayment(decimal amount)
    {
        // Payment logic
        return true;
```

```
    }
}
```

- If you **target .NET Standard 2.0**, this library can be used in:
  - **ASP.NET Core web app**
  - **Xamarin mobile app**
  - **.NET Framework desktop app**

This avoids rewriting the library for each platform.

Q61: What is the difference between .NET Core and .NET Framework?

Ans-

Feature	.NET Framework	.NET Core
Definition	A Windows-only framework for building desktop and web applications.	A cross-platform, open-source framework for building modern applications (Web, API, Cloud, Microservices).
Platform	Runs only on Windows	Runs on Windows, Linux, macOS
Open Source	No	Yes, open source on GitHub
Deployment	Installed on Windows machine	Side-by-side deployment per application
Performance	Good	High performance, optimized for modern apps
App Types	Web apps (ASP.NET), Desktop apps (WinForms, WPF)	Web apps (ASP.NET Core), APIs, Microservices, Cloud, Cross-platform apps

Real-World Example

- .NET Framework:
  - Legacy desktop applications like internal company tools on Windows.
  - ASP.NET WebForms websites.
- .NET Core:
  - Modern web applications, REST APIs, microservices.
  - Cross-platform apps that run on Linux servers or cloud.
  - Examples: Netflix, StackOverflow, GoDaddy APIs.

Q62: What is the role of Program.cs file in ASP.NET Core?

Ans-The Program.cs file is the entry point of an ASP.NET Core application. It is responsible for:

1. Configuring the web host
2. Setting up services and middleware
3. Starting the application

In short: Program.cs tells the application how to start, configure, and run.

How it Works

In ASP.NET Core 6/7, a typical Program.cs looks like this:

```
var builder = WebApplication.CreateBuilder(args);


// Add services to the container
builder.Services.AddControllers();
builder.Services.AddDbContext<MyDbContext>();


var app = builder.Build();


// Configure middleware
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}


app.UseHttpsRedirection();
```

Codewithindu

```
app.UseAuthorization();


app.MapControllers();


app.Run();
```

Step-by-Step Explanation

1. Create Builder

```
var builder = WebApplication.CreateBuilder(args);
```

- Sets up the host for the application.
- Configures environment variables, logging, and configuration.

2. Configure Services

```
builder.Services.AddControllers();
```

- Registers services for Dependency Injection (DI).
- Example: DbContext, Identity, HttpClient.

3. Build the Application

```
var app = builder.Build();
```

- Creates the application pipeline (middleware setup).

4. Configure Middleware

```
app.UseHttpsRedirection();

app.UseAuthorization();
```

- Middleware handles request processing (like authentication, routing, exception handling).

5. Map Endpoints

```
app.MapControllers();
```

codewithindu

- Connects controllers or endpoints to handle incoming HTTP requests.

6. Run the Application

```
app.Run();
```

- Starts the Kestrel web server and begins listening for requests.

.NET Core- Dependency Injection

Q63: What is Dependency Injection (DI)?

Ans-**Dependency Injection (DI)** is a design pattern used to **achieve Inversion of Control (IoC)** between classes and their dependencies.

- Instead of a class **creating its own dependencies**, they are **provided from the outside** (injected).
- This makes the code **loosely coupled, testable, and maintainable**.

In simple words: "**Give an object what it needs, don't let it create it.**"

Types of Dependency Injection in ASP.NET Core

1. **Constructor Injection** (most common)
2. **Property Injection**
3. **Method Injection**

Example in ASP.NET Core

Suppose you have a service EmailService used by a UserController.

```
// Dependency

public interface IEmailService

{

    void SendEmail(string to, string subject);

}


public class EmailService : IEmailService
```

```
{
    public void SendEmail(string to, string subject)
    {
        Console.WriteLine($"Email sent to {to} with subject {subject}");
    }
}

// Consumer
public class UserController
{
    private readonly IEmailService _emailService;

    // Constructor Injection
    public UserController(IEmailService emailService)
    {
        _emailService = emailService;
    }

    public void RegisterUser(string email)
    {
        // Business logic...
        _emailService.SendEmail(email, "Welcome!");
    }
```

Codewithindu

```
}
```

Register DI in Program.cs:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddScoped<IEmailService, EmailService>();
```

ASP.NET Core **injects the dependency** automatically when UserController is created.

Q64: How to implement Dependency Injection in .NET Core?

Ans-**Step 1: Define a Service Interface**

```
public interface IEmailService
{
    void SendEmail(string to, string subject);
}
```

- This is the **abstraction**. Always code to **interfaces** rather than concrete classes.

Step 2: Implement the Service

```
public class EmailService : IEmailService
{
    public void SendEmail(string to, string subject)
    {
        Console.WriteLine($"Email sent to {to} with subject '{subject}'");
    }
}
```

- This is the **concrete implementation** that performs the actual work.

Step 3: Register the Service in Program.cs

```
var builder = WebApplication.CreateBuilder(args);


// Add services to DI container

builder.Services.AddScoped<IEmailService, EmailService>(); // Scoped lifetime
```

ASP.NET Core provides three main lifetimes for services:

Lifetime	Description	Example Use Case
Transient	New instance every time requested	Lightweight services
Scoped	Same instance within a request, new per request	DbContext for each HTTP request
Singleton	Same instance throughout the application	Logging, configuration

---

Step 4: Inject the Service into a Consumer

```
public class UserController : ControllerBase
{
    private readonly IEmailService _emailService;


    // Constructor Injection
    public UserController(IEmailService emailService)
    {
        _emailService = emailService;
    }


    [HttpPost("register")]
```

Codewithindu

```
    public IActionResult RegisterUser(string email)
    {
        // Business logic...
        _emailService.SendEmail(email, "Welcome!");
        return Ok("User registered successfully!");
    }
}
```

- The **DI container automatically provides the EmailService** whenever UserController is instantiated.

Step 5: Use the Controller

- When a client calls POST /register, ASP.NET Core automatically injects EmailService into UserController.
- You don't need to manually create EmailService.

Q65: What are the Advantages of Dependency Injection in .NET Core?

Ans-**Dependency Injection (DI)** is a design pattern where a class receives its dependencies from an external source rather than creating them internally.

In ASP.NET Core, DI is **built-in** and widely used to manage services like DbContext, EmailService, Logging, etc.

Advantages of Dependency Injection

Advantage	Explanation	Example
1. Loose Coupling	Classes are **not tightly bound** to specific implementations. You can easily switch implementations.	Replace EmailService with MockEmailService for testing.

Advantage	Explanation	Example
2. Improved Testability	Dependencies can be **mocked or stubbed** in unit tests.	Unit test UserController with a mock IEmailService.
3. Reusability	Services can be **reused across multiple classes** without re-instantiation.	DbContext or LoggingService shared across multiple controllers.
4. Maintainability	Centralized service registration makes **code easier to maintain and extend**.	Adding PaymentService only in Program.cs without modifying controllers.
5. Scalability	Supports **different lifetimes** (Transient, Scoped, Singleton) to optimize performance and resource usage.	Scoped DbContext per HTTP request, Singleton Logger across app.
6. Separation of Concerns	Each class focuses on its **own responsibility**, dependencies are injected.	Controller handles HTTP requests, service handles business logic.
7. Built-in Support in ASP.NET Core	No need for third-party DI frameworks; fully integrated.	Use builder.Services.AddScoped<IService, Service>() in Program.cs.

.NET Core- Service Lifetimes, Middleware & Hosting

Q66: What are the Types of Service Lifetimes in ASP.NET Core?

Ans-In **ASP.NET Core**, when you register a service in the **Dependency Injection (DI) container**, you must specify its **lifetime**. The lifetime defines **how long the service instance exists**.

1️⃣ Transient Lifetime

- **Definition:** A **new instance of the service is created every time** it is requested from the DI container.
- **Use Case:** Lightweight, stateless services.
- **Registration Example:**

```
builder.Services.AddTransient<IEmailService, EmailService>();
```

- **Example in Action:**

```
var service1 = provider.GetService<IEmailService>();

var service2 = provider.GetService<IEmailService>();

// service1 != service2 (different instances)
```

2️⃣ Scoped Lifetime

- **Definition:** A **single instance is created per HTTP request**.
- **Use Case:** Services that need to **share data within the same request** but not across requests, e.g., DbContext.
- **Registration Example:**

```
builder.Services.AddScoped<IOrderService, OrderService>();
```

- **Example in Action:**

```
var service1 = provider.GetService<IOrderService>(); // during request

var service2 = provider.GetService<IOrderService>(); // same request

// service1 == service2 (same instance within request)
```

3️⃣ Singleton Lifetime

- **Definition:** A **single instance is created for the entire lifetime of the application**.
- **Use Case:** Shared services across all requests, e.g., logging, configuration.

- **Registration Example:**

```
builder.Services.AddSingleton<ILoggerService, LoggerService>();
```

- **Example in Action:**

```
var service1 = provider.GetService<ILoggerService>();

var service2 = provider.GetService<ILoggerService>();

// service1 == service2 (same instance across app)
```

Q67: What are AddSingleton, AddScoped, and AddTransient methods in ASP.NET Core?

Ans-In **ASP.NET Core**, when we register a service in the **Dependency Injection (DI) container**, we use these methods to define **the lifetime of the service instance**.

Method	Lifetime	Description
AddTransient	Transient	A **new instance** is created every time the service is requested.
AddScoped	Scoped	A **single instance per HTTP request** is created and reused within that request.
AddSingleton	Singleton	A **single instance for the entire application** is created and reused across all requests.

Q68: What is Middleware in ASP.NET Core? What is Custom Middleware?

Ans-Middleware is a **software component** that is assembled into the **HTTP request pipeline** to **handle requests and responses**.

- Each **HTTP request** passes through multiple middleware components before reaching the controller.
- Middleware can **process requests, modify responses, call the next middleware, or short-circuit the pipeline**.

Common Built-in Middleware in ASP.NET Core

Middleware	Purpose
UseRouting()	Routes requests to endpoints

Middleware	Purpose
UseAuthentication()	Handles user authentication
UseAuthorization()	Handles user authorization
UseStaticFiles()	Serves static files like CSS, JS, images
UseExceptionHandler()	Handles exceptions globally
UseCors()	Enables Cross-Origin Requests

Pipeline Example in Program.cs:

```
var app = builder.Build();


if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();


app.MapControllers();
app.Run();
```

Flow Diagram:

Incoming Request → Middleware1 → Middleware2 → Middleware3 → Controller → Response

What is Custom Middleware

Custom middleware is a **user-defined middleware** to handle specific tasks not covered by built-in middleware.

Example: Logging request details:

```
public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;

    public RequestLoggingMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        Console.WriteLine($"Request Path: {context.Request.Path}");
        await _next(context); // Call the next middleware
    }
}

// Extension method to make it easy to use
public static class RequestLoggingMiddlewareExtensions
{
```

Codewithindu

```
    public static IApplicationBuilder UseRequestLogging(this IApplicationBuilder
builder)
    {
        return builder.UseMiddleware<RequestLoggingMiddleware>();
    }
}
```

Register custom middleware in Program.cs:

```
app.UseRequestLogging();
```

- Every incoming HTTP request now **logs its path** before reaching the controller.

Q69: How is ASP.NET Core Middleware different from HttpModule?

Ans-**ASP.NET Core Middleware**

Middleware in **ASP.NET Core** is a **component in the request pipeline** that handles HTTP requests and responses.

- Middleware is **lightweight, modular, and part of the pipeline configuration** in Program.cs.
- Middleware can **process requests, responses, and call the next component** using RequestDelegate.

Key Points:

- Configured in **Program.cs** using app.Use...() methods.
- **Order matters:** Executes in the order it is added.
- Can be **custom or built-in**.
- Part of **cross-platform, modern ASP.NET Core** architecture.

Example:

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Before request");
```

```
    await next.Invoke();  // Call next middleware
    Console.WriteLine("After request");
});
```

2️⃣ASP.NET HttpModule (Classic ASP.NET)

HttpModule is a **component in classic ASP.NET (Framework)** that handles **events during the HTTP request pipeline**.

- It **subscribes to request/response events** like BeginRequest, EndRequest, AuthenticateRequest.
- Requires **web.config registration**.

Key Points:

- Configured in **web.config**.
- Event-driven: Executes when a specific HTTP event occurs.
- Part of **Windows-only ASP.NET Framework**.

Example (web.config registration):

```
<system.webServer>
  <modules>
    <add name="CustomModule" type="MyNamespace.CustomModule"/>
  </modules>
</system.webServer>
```

Custom HttpModule Example:

```
public class CustomModule : IHttpModule
{
    public void Init(HttpApplication context)
    {
        context.BeginRequest += (src, args) =>
```

```
        {
            Console.WriteLine("Begin Request");
        };
    }

    public void Dispose() { }
}
```

Q70: What is Run(), Use(), and Map() method in ASP.NET Core?

Ans-**1. Use() Method**

- Use() adds a **middleware component to the pipeline** that can **perform operations before and after calling the next middleware**.
- It **must call next()** to pass the request to the next middleware.

Syntax Example:

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Before next middleware");
    await next.Invoke();  // Call next middleware
    Console.WriteLine("After next middleware");
});
```

Key Points:

- Can perform actions **before and after** the next middleware.
- **Chained middleware** forms the request pipeline.

2. Run() Method

Codewithindu

- Run() adds a **terminal middleware** that **does not call the next middleware**.
- It **ends the request pipeline**.

Syntax Example:

```
app.Run(async (context) =>
{
    await context.Response.WriteAsync("Hello from Run!");
});
```

Key Points:

- **Terminal middleware** – stops pipeline execution.
- Used for **short-circuiting requests**, like returning a response immediately.

3️⃣ Map() Method

Definition:

- Map() is used to **branch the request pipeline** based on the request path.
- It allows **specific middleware to run only for a certain URL**.

Syntax Example:

```
app.Map("/admin", adminApp =>
{
    adminApp.Run(async context =>
    {
        await context.Response.WriteAsync("Welcome Admin!");
    });
});


app.Map("/user", userApp =>
```

```
{
    userApp.Run(async context =>
    {
        await context.Response.WriteAsync("Welcome User!");
    });
});
```

Q71What is Kestrel? What is the difference between Kestrel and IIS?

Ans-**Kestrel** is a **cross-platform, lightweight, built-in web server** used by **ASP.NET Core applications**.

It is the **default web server** for ASP.NET Core
Works on **Windows, Linux, and macOS**
Very fast and optimized for **asynchronous I/O**
Can handle **millions of requests per second**

Simple Explanation:
Kestrel is the small but very fast web server **inside** your ASP.NET Core app.
It listens to HTTP requests and sends responses back.

Difference Between Kestrel and IIS

Feature	**Kestrel**	**IIS (Internet Information Services)**
Platform	Cross-platform (Win, Linux, macOS)	Windows only
Type	Lightweight, fast web server	Full-featured enterprise web server
Default Use	Default server for ASP.NET Core apps	Traditional ASP.NET + reverse proxy for ASP.NET Core
Performance	Very fast, high-performance	Slower than Kestrel but very stable for enterprise

Feature	Kestrel	IIS (Internet Information Services)
Security Features	Minimal built-in security	Advanced security (SSL, filtering, authentication)
Reverse Proxy	Commonly used **behind** IIS/Nginx	Acts as **reverse proxy** for Kestrel
Hosting Model	Self-hosted inside .NET Core	Windows service with worker process (w3wp.exe)
Suited For	Cross-platform apps, microservices	Enterprise Windows hosting, large websites

SOLID Principles

Q72.What are SOLID Principles?

Ans-**SOLID Principles**

SOLID has **5 rules** to write clean and good object-oriented code.

1) S → Single Responsibility Principle (SRP)

A class should have **only one responsibility** and **one reason to change**.

One class = One job.

Example-

```
class Invoice
{
    public void SaveInvoice() { }
}


class EmailService
```

```
{
    public void SendEmail() { }
}
```

2) O — Open/Closed Principle (OCP)

Code should be **open for extension**, but **closed for modification**.

Example

You want to add new payment types without changing old code.

```
interface IPayment
{
    void Pay();
}

class CreditCard : IPayment
{
    public void Pay() {}
}


class UPI : IPayment
{
    public void Pay() {}
}
```

➞ Tomorrow if you add **PayPal**, just create a new class. No need to modify existing code.

3) L — Liskov Substitution Principle (LSP)

Definition: A child class must be completely replaceable for its parent class **without breaking the program**.

Simple Meaning:

Child should behave like parent.

Example-

```
class Bird { }


class FlyingBird : Bird
{
    public void Fly() { }
}
class Ostrich : Bird
{
    // Does not have Fly()
}
```

Codewithindu

4) I — Interface Segregation Principle (ISP)

Definition:

Don't force a class to implement methods it doesn't need. Use **small, specific interfaces**.

Clients should not be forced to depend on interfaces they do not use. Use **small, specific interfaces** instead of one large interface.

Very Simple Meaning:

Don’t force extra methods; keep interfaces small.

Example-

```
interface IWork
{
    void Work();
}

interface IEat
{
    void Eat();
}

class Robot : IWork
{
    public void Work() { }
}
```

5) D — Dependency Inversion Principle (DIP)

Proper Definition:

High-level modules should depend on **abstractions (interfaces)**, not on **concrete implementations**.

Simple Meaning:

Depend on interface, not class.

Example-

```
interface IMessageService
{
    void Send();
}

class EmailService : IMessageService
{
    public void Send() { }
}

class Notification
{
    private readonly IMessageService _service;

    public Notification(IMessageService service)
    {
        _service = service;
    }
}
```

Q73.How SOLID Principles are different from Design Patterns?

Ans-**SOLID Principles**

SOLID is a **set of 5 rules** that guide you to write **clean, maintainable, and flexible code**.

They tell you **how to structure your classes and methods**.

Design Patterns

Design Patterns are **reusable solutions** to common programming problems.

They tell you **how to solve specific problems** using predefined templates.

Difference (Very Easy to Remember)

SOLID Principles	**Design Patterns**
Rules/Guidelines for writing good code	**Ready-made solutions** to common problems
Helps in **code quality, structure, flexibility**	Helps in **problem-solving and reusable architecture**
Generic principles	Concrete implementation ideas
Always used in OOP	Used when a specific problem occurs
Example: SRP, OCP, LSP…	Example: Singleton, Factory, Repository…

Simple Real-World Example

SOLID Principle Example

SRP → "One class should do one job."

If you create a class:

```
class InvoiceService { Save(); Calculate(); Email(); }
```

SOLID says → ✕ wrong
Split them into separate classes.

Design Pattern Example

Factory Pattern → Used when you want to create objects without exposing creation logic.

```
public class PaymentFactory
```

```
{
    public static IPayment CreatePayment(string type)
    {
        if (type == "UPI") return new UPIPayment();
        if (type == "Card") return new CardPayment();
        return null;
    }
}
```

Relationship Between Both

SOLID prepares your code

→ to easily apply **design patterns**.

Design Patterns follow SOLID

→ almost every design pattern depends on SOLID principles.

Q74-What is Single Responsibility Principle?

Ans-**Single Responsibility Principle (SRP)** says that:

A class should have only one responsibility and only one reason to change.

Meaning:
A class should do **only one job**, not multiple jobs.

Simple Explanation

If a class does too many things, changing one feature may break another. So we divide the work into **small focused classes**.

Real-World Example

Think of a **Restaurant**:

- **Chef** cooks food
- **Waiter** serves food
- **Cashier** handles billing

Each role has **one responsibility**.

If the waiter also handles billing + cooking → everything becomes confusing.

Example-

```
class InvoiceService
{
    public void SaveInvoice() { }
}


class EmailService
{
    public void SendEmail() { }
}
```

Each class now has **one responsibility only**.

SRP means a class should only do one job and have only one reason to change. It makes the system easy to maintain, test, and extend.

Q75-What is Open-Closed Principle (OCP)?

Ans-**Open-Closed Principle (OCP)** states:

Classes should be OPEN for extension but CLOSED for modification

Meaning:
You should be able to **add new features** without **changing existing code**.

Very Simple Explanation

Don’t edit old code
Add new code to extend behavior

This avoids bugs and makes the system stable.

Real-World Example

Consider a **Payment System**:

Today app supports:

- Credit Card
  Tomorrow you need:
- UPI
- NetBanking

You should **not change the old payment code**. Instead, **add new payment classes**.

Simple C# Example

Use **interface** → Add new classes without modifying old ones:

```
interface IPayment

{

   void Pay();

}


class CardPayment : IPayment

{

   public void Pay() { }

}


class UpiPayment : IPayment
```

```
{
    public void Pay() { }
}
```

Now PaymentService remains **unchanged**:

```
class PaymentService
{
    public void Process(IPayment payment)
    {
        payment.Pay();
    }
}
```

➞ To add a new payment:
Just create a new class NetBankingPayment → No changes in existing classes.

One-Line Interview Answer

Open-Closed Principle states that software should be open for extension but closed for modification. We can add new features by writing new code without modifying existing, stable code.

Q76-What is Liskov Substitution Principle (LSP)?

Ans-Liskov Substitution Principle says:

A child class should be able to replace its parent class without breaking the program.

In simple words:

- Wherever you use a parent class, you should be able to use any child class without errors, unexpected behavior, or changing the logic.

Example- Split birds into **FlyingBird** and **NonFlyingBird**.

```
public abstract class Bird
{
    public abstract void Eat();
}

public abstract class FlyingBird : Bird
{
    public abstract void Fly();
}

public class Sparrow : FlyingBird
{
    public override void Eat() {}
    public override void Fly() {}
}

public class Penguin : Bird
{
    public override void Eat() {}
}
```

Q77-What is DRY Principle?

Ans-**DRY = Don’t Repeat Yourself**

It means:

Don’t write the same code or logic again and again. Keep it in one place and reuse it everywhere.

Why DRY

- Reduces duplicate code
- Makes code clean and maintainable
- When one change is needed → you update at only one place
- Avoids bugs and inconsistency

Design Patterns

Q78-What Are Design Patterns?

Ans-Design Patterns are reusable solutions to common software problems.

They are **not code**, they are **proven ways (templates) to structure code** to solve a repeated problem.

Codewithindu

Think of them like:

Ready-made blueprints that guide you in writing clean, flexible, and maintainable code.

We Need Design Patterns

Because in software development, you will face many common challenges:

- Object creation
- Object communication
- Code structure
- Managing complexity
- Avoiding tightly-coupled code

Design patterns solve these problems in **standard, time-tested ways**.

What Problems Do Design Patterns Solve

Here are the key problems design patterns address:

❌Problem: Code is difficult to change (tight coupling)

✓ **Solution: Patterns like Dependency Injection, Strategy, Adapter**

Example:
If your payment system directly uses PayPal class, then switching to Stripe becomes painful.
Using Strategy Pattern → You can switch payment providers easily.

2️⃣ Problem: Object creation is complex

Solution: Factory Pattern, Builder Pattern, Singleton Pattern

Example:
When creating a complex object with many optional parameters → Builder Pattern simplifies it.

3️⃣ Problem: Repeated business logic

Solution: Template Method, Strategy

Example:
Different types of discounts but same calculation flow → Template Method fits perfectly.

4️⃣ Problem: Code is duplicated or scattered

Solution: Facade, Mediator

Example:
Instead of calling 10 different services, Facade creates a single simplified interface.

5️⃣ Problem: Hard to maintain large applications Solution:

All design patterns help in maintainability Patterns

reduce code complexity and make the system clean. Q79-

What Are the Types of Design Patterns?

Ans-Design Patterns are mainly divided into **three categories**:

1. Creational Design Patterns

These patterns deal with **object creation** in a flexible and reusable way.

Why?

To avoid:

- long constructors
- complex object creation
- tightly coupled code

Examples (5):

1. **Singleton** – Only one instance of a class.
2. **Factory Method** – Creates objects without exposing creation logic.
3. **Abstract Factory** – Creates families of related objects.
4. **Builder** – Builds complex objects step-by-step.
5. **Prototype** – Clones existing objects.

2. Structural Design Patterns

These patterns deal with **class and object composition** — how objects are connected.

codewithindu

Why?

To make code:

- more flexible
- better organized
- easy to extend

Examples (7):

1. **Adapter** – Converts one interface to another.
2. **Facade** – Provides a simple interface to complex systems.
3. **Proxy** – Control access to another object.
4. **Decorator** – Add new features to objects at runtime.
5. **Composite** – Treat a group of objects as a single object.
6. **Bridge** – Separate abstraction from implementation.
7. **Flyweight** – Reduce memory usage by sharing objects.

3. Behavioral Design Patterns

These patterns deal with **object interaction and communication**.

Why?

To make communication:

- flexible
- clean
- loosely coupled

Examples (9):

1. **Strategy** – Switch algorithms at runtime.
2. **Observer** – Notify multiple objects automatically.
3. **Command** – Encapsulate actions as objects.
4. **Iterator** – Access collection elements one by one.
5. **Mediator** – Central communication controller.
6. **Template Method** – Define the skeleton of an algorithm.
7. **State** – Change behavior based on internal state.
8. **Chain of Responsibility** – Pass request along a chain.
9. **Interpreter** – Implements a language grammar.

Q80-What are Creational Design Patterns?

Ans-**Creational Design Patterns** are design patterns that focus on **how objects are created** in a flexible, reusable, and maintainable way — without exposing the object creation logic to the client.

Instead of using **new keyword everywhere**, creational patterns give smarter ways to create objects.

Types of Creational Design Patterns

Pattern	**Purpose**	**Real-world Example**
1. Singleton	Only one instance of a class	Printer Spooler, Logger
2. Factory Method	Creates objects without exposing creation logic	Pizza Store deciding pizza type
3. Abstract Factory	Creates families of related objects	UI themes (Dark/Light)
4. Builder	Builds complex objects step-by-step	Building a House (walls → roof → paint)
5. Prototype	Cloning already created objects	Copy-Paste in Word, Game characters

Creational design patterns provide flexible ways to create objects without tightly coupling the code. They hide the complexity of object creation.

Simple C# Examples

1. Singleton Pattern

```
public class Logger
{
    private static Logger _instance;

    private Logger() { }

    public static Logger Instance
    {
        get
        {
            if (_instance == null)
                _instance = new Logger();

            return _instance;
        }
    }

    public void Log(string message)
    {
```

Codewithindu

```
        Console.WriteLine(message);
    }
}
```

Usage:

```
Logger.Instance.Log("Hello");
```

2. Factory Method

```
public abstract class Shape
{
    public abstract void Draw();
}

public class Circle : Shape
{
    public override void Draw() => Console.WriteLine("Circle");
}

public class Square : Shape
{
    public override void Draw() => Console.WriteLine("Square");
}

public class ShapeFactory
{
```

Codewithindu

```
    public static Shape CreateShape(string type)
    {
        return type switch
        {
            "circle" => new Circle(),
            "square" => new Square(),
            _ => null
        };
    }
}
```

3. Builder Pattern

```
public class House
{
    public string Walls { get; set; }
    public string Roof { get; set; }
}


public class HouseBuilder
{
    private House house = new House();

    public HouseBuilder BuildWalls()
    {
```

Codewithindu

```
        house.Walls = "Brick Walls";
        return this;
    }

    public HouseBuilder BuildRoof()
    {
        house.Roof = "Cement Roof";
        return this;
    }

    public House Build() => house;
}
```

4. Prototype Pattern

```
public class Employee : ICloneable
{
    public string Name { get; set; }

    public object Clone()
    {
        return this.MemberwiseClone();
    }
}
```

Q81-What are Structural Design Patterns?

Ans-**Structural Design Patterns** focus on **how classes and objects are combined or structured** to build larger, flexible, and maintainable systems.

They help in organizing code so that:

- It becomes easier to extend
- Objects can work together smoothly
- Complex structures look simple

Types of Structural Design Patterns

Pattern	Purpose	Simple Real-World Example
1. Adapter	Converts one interface into another	Mobile charger adapter
2. Facade	Provides a simple interface to complex system	TV remote controlling entire system
3. Decorator	Add new features at runtime without modifying class	Adding toppings on a pizza
4. Proxy	Provides a placeholder or controller for another object	ATM machine acting as bank proxy
5. Composite	Treat a group of objects like a single object	Folder with files and subfolders
6. Bridge	Separates abstraction from implementation	Remote control (abstraction) + TV (implementation)
7. Flyweight	Share common objects to save memory	Chess pieces with shared shapes

Simple C# Examples

Adapter Pattern

Adapts one interface to another.

```
public interface INewCharger
{
    void Charge();
```

```
}


public class OldCharger
{
    public void OldCharge() => Console.WriteLine("Charging with old charger");
}


public class ChargerAdapter : INewCharger
{
    private OldCharger oldCharger = new OldCharger();

    public void Charge()
    {
        oldCharger.OldCharge();
    }
}
```

Q82.What Are Behavioral Design Patterns?

Ans-**Behavioral Design Patterns** define **how objects communicate and interact with each other**.
They focus on:

- The **responsibility** of objects
- **Communication** between objects
- **How behavior flows** in the system

In short:

Behavioral patterns help objects talk to each other in a flexible, maintainable way.

Types of Behavioral Design Patterns

Pattern	Purpose	Real-World Example
1. Strategy	Choose behavior at runtime	Choosing payment method: UPI/Card/Cash
2. Observer	Notify many objects when one changes	YouTube Subscribe Notification
3. Command	Encapsulate requests as objects	Undo/Redo command in apps
4. Chain of Responsibility	Pass request until someone handles it	Customer Support Call Routing
5. Iterator	Sequentially access elements	For-each loop
6. Mediator	Central controller to manage communication	Air Traffic Control
7. Memento	Save/restore object state	Ctrl+Z (Undo)
8. State	Change behavior based on state	Fan speed: Low/Medium/High
9. Template Method	Define algorithm skeleton	Cooking steps with different recipes
10. Visitor	Add new operations without modifying objects	Tax calculation on products
11. Interpreter	Evaluate sentences/expressions	SQL query interpreter

Q83. What is Singleton Design Pattern?

Ans- Singleton Design Pattern ensures that:

1. **Only one instance** of a class is created
2. The instance is **globally accessible** throughout the application

This means **you cannot create multiple objects** of that class.

Why Singleton is used?

Because sometimes you need **one and only one** object for the entire application.

Example:

- Only **one logger** is needed to write application logs

- Only **one database connection manager**
- Only **one configuration reader**
- Only **one cache manager**

If multiple objects are created → inconsistency, performance issues.

C# Singleton Example

```
public sealed class Logger
{
    private static Logger _instance = null;
    private static readonly object _lock = new object();

    // Private constructor - prevents creating object from outside
    private Logger() { }

    public static Logger Instance
    {
        get
        {
            lock (_lock)            // thread-safe
            {
                if (_instance == null)
                {
                    _instance = new Logger();
                }
                return _instance;
```

Codewithindu

```
        }
    }
}

public void Log(string message)
{
    Console.WriteLine(message);
}
}
```

Usage:

```
Logger.Instance.Log("App started");
```

No matter how many times you call Logger.Instance, you always get **the same single object**.

SQL- Joins & Indexes

Q84-What are Joins in SQL?

Ans-SQL Joins are used to **combine rows from two or more tables** based on a related column between them.

In simple words: **Joins help you fetch data from multiple tables as if it is one table.**

Joins are used

- Data is stored in **multiple related tables** (Normalization).
- We often need **combined information** from two or more tables.
- Joins help us **avoid duplicate data** and make queries efficient.

Types of SQL Joins

Join Type	Definition	Real-World Example
1. INNER JOIN	Returns rows **common** in both tables	Students with registered courses
2. LEFT JOIN (or LEFT OUTER JOIN)	Returns all rows from **left table**, matching from right table, NULL if no match	All students + their marks (even if some don't have marks)
3. RIGHT JOIN (or RIGHT OUTER JOIN)	Returns all rows from **right table**, matching from left table, NULL if no match	All courses + enrolled students (even if some courses have no students)
4. FULL JOIN (or FULL OUTER JOIN)	Returns **all rows** from both tables, NULL if no match	Complete list of students and courses
5. CROSS JOIN	Returns **all possible combinations** of rows	All sizes × all colors of a product
6. SELF JOIN	Table joined with **itself**	Employees with their manager (same table)

Codewithindu

Real-Life Example

Tables:Students

StudentId	Name
1	Alice
2	Bob

Marks

StudentId	Marks
1	90
3	80

INNER JOIN

```
SELECT Students.Name, Marks.Marks
FROM Students
INNER JOIN Marks
```

ON Students.StudentId = Marks.StudentId;

Result:

Name	Marks
Alice	90

Only common StudentId (1) appears.

2. LEFT JOIN

SELECT Students.Name, Marks.Marks

FROM Students

LEFT JOIN Marks

ON Students.StudentId = Marks.StudentId;

Result:

Name	Marks
Alice	90
Bob	NULL

All students, marks if available.

3. RIGHT JOIN

SELECT Students.Name, Marks.Marks

FROM Students

RIGHT JOIN Marks

ON Students.StudentId = Marks.StudentId;

Result:

Name	Marks
Alice	90
NULL	80

All marks, student name if available.

4. FULL OUTER JOIN

```
SELECT Students.Name, Marks.Marks
FROM Students
FULL OUTER JOIN Marks
ON Students.StudentId = Marks.StudentId;
```

Result:

Name	Marks
Alice	90
Bob	NULL
NULL	80

Combines LEFT and RIGHT JOIN results.

5. CROSS JOIN

```
SELECT Students.Name, Marks.Marks
FROM Students
CROSS JOIN Marks;
```

Result:

Name	Marks
Alice	90
Alice	80
Bob	90
Bob	80

All combinations of rows.

6. SELF JOIN

Codewithindu

```
SELECT e1.Name AS Employee, e2.Name AS Manager
FROM Employees e1
INNER JOIN Employees e2
ON e1.ManagerId = e2.EmployeeId;
```

Used when a table has **hierarchical relationship with itself**.

Q85. What are the types of Joins in SQL Server?

Ans-Types of Joins in SQL Server

Join Type	Definition	Example Use Case
1. INNER JOIN	Returns rows **that have matching values in both tables**	Students who have marks recorded
2. LEFT JOIN (LEFT OUTER JOIN)	Returns **all rows from the left table** and matching rows from the right table; NULL if no match	List of all students with their marks (even if some students don't have marks)
3. RIGHT JOIN (RIGHT OUTER JOIN)	Returns **all rows from the right table** and matching rows from the left table; NULL if no match	List of all courses with enrolled students (even if some courses have no students)
4. FULL OUTER JOIN	Returns **all rows from both tables**; NULL where there is no match	Complete list of students and marks, including missing data
5. CROSS JOIN	Returns **all possible combinations of rows** from both tables	Generating combinations of products and sizes
6. SELF JOIN	Joins a table **with itself** using aliases	Employee and their manager within the same table

Q86-What is Self-Join?

Ans-A **Self-Join** is a join in which a table is **joined with itself**.

In simple words: **A table can act as both the "left" and "right" table in the same query** by using **aliases**.

Why Use Self-Join

- To compare rows within the **same table**
- To find **hierarchical relationships**
- To fetch related data stored in the **same table**

Example: Employee and Manager

EmployeeId	Name	ManagerId
1	Alice	NULL
2	Bob	1
3	Charlie	1
4	David	2

We want: **Employee name with their Manager name**

The data is in **the same table**, so we use a Self-Join.

SQL Query Example

```
SELECT e1.Name AS Employee, e2.Name AS Manager
FROM Employees e1
LEFT JOIN Employees e2
ON e1.ManagerId = e2.EmployeeId;
```

Result

Employee	Manager
Alice	NULL
Bob	Alice
Charlie	Alice
David	Bob

A Self-Join is a join in which a table is joined with itself using aliases, mainly to find relationships within the same table, like employee-manager or parent-child hierarchy.

Q87.What are Indexes in SQL Server?

Ans-An **Index** in SQL Server is a **database object** that **improves the speed of data retrieval** from a table.
It works like an **index in a book**, allowing SQL Server to **quickly find rows without scanning the entire table**.

Why Indexes Are Used

- To **speed up SELECT queries**
- To **improve JOIN performance**
- To **enforce uniqueness** (in case of unique indexes)
- To **reduce query execution time** in large tables

Without indexes, SQL Server performs a **full table scan**, which is slower.

Types of Indexes

Index Type	Definition	Example Use Case
1. Clustered Index	Determines physical order of data in table. Only one clustered index per table	Primary key column
2. Non-Clustered Index	Separate structure pointing to data rows. Multiple non-clustered indexes allowed	Searching by name or email
3. Unique Index	Ensures all values are unique in a column	Employee email or username
4. Composite Index	Index on multiple columns	Searching by FirstName + LastName
5. Full-Text Index	Index for text-based searches	Searching keywords in documents
6. Filtered Index	Index for a subset of rows	Active customers only

Q88-What is Clustered index?

Ans-A Clustered Index is a type of index that sorts and stores the data rows in the table based on the key column(s).

In simple words: The table itself is organized physically in the order of the clustered index.

Important: A table can have only one clustered index because the data can be stored in only one order.

Why Use Clustered Index?

- Speeds up **queries that use range searches** (e.g., BETWEEN, >, <)
- Speeds up **sorting and grouping** (ORDER BY, GROUP BY)
- Efficient for **primary key columns**

Real-World Analogy

Think of a **telephone directory**:

- Pages are **sorted by last name**.
- If you want "Alice Johnson," you **directly go to the J-section**.
- The **physical order of pages matches the sorting** → similar to clustered index.

SQL Example

Create a Clustered Index

```
CREATE CLUSTERED INDEX IX_EmployeeID
ON Employees(EmployeeId);
```

Query Benefit

```
SELECT * FROM Employees
WHERE EmployeeId = 1001;
```

SQL Server directly navigates to the data row because the table is **physically sorted by EmployeeId**.

A Clustered Index is an index that sorts and stores the table's data rows physically based on the key column. Each table can have only one clustered index.

Q89.What is a Non-Clustered Index?

Ans-A **Non-Clustered Index** is a type of index that **creates a separate structure from the table** to store the **index key and a pointer to the actual data row**.

In simple words: The **table's data is not sorted physically**, but the index helps SQL Server **quickly locate rows**.

Important: A table can have **multiple non-clustered indexes**.

Why Use Non-Clustered Index?

- Speeds up **queries on columns other than the primary key**
- Improves **search performance** without reordering the table
- Useful for **columns frequently used in WHERE, JOIN, ORDER BY**

Sql Example

Create a Non-Clustered Index

```
CREATE NONCLUSTERED INDEX IX_EmployeeName
ON Employees(Name);
```

Query Using Non-Clustered Index

```
SELECT * FROM Employees
WHERE Name = 'Alice';
```

SQL Server uses the **index to locate the row pointer**, then retrieves the full data row.

A Non-Clustered Index is an index stored separately from the table that contains key values and pointers to the actual data rows, allowing fast data retrieval without changing the table's physical order.

Q90-What is the difference between Clustered and Non-Clustered index?

Ans-**Clustered vs Non-Clustered Index**

Feature	**Clustered Index**	**Non-Clustered Index**
Definition	Sorts and stores the **actual table data rows** physically in the order of the key	Stores a **separate structure** containing key values and pointers to the actual table rows
Physical Order	Table rows are **physically sorted**	Table rows are **not sorted**; index points to data
Number per Table	Only **1 per table**	Can have **multiple per table**
Storage	Stored in the **data pages** of the table	Stored **separately** from table data

Feature	Clustered Index	Non-Clustered Index
Best Use Case	Columns used in **primary key, range queries, ORDER BY**	Columns frequently used in **WHERE, JOIN, ORDER BY** but not primary key
Performance	Faster for **range queries** and sorting	Faster for **lookup queries** on non-primary key columns
Insert/Update Impact	Slower because data may need to be **reordered**	Slight overhead because index must also be **updated**
Example	EmployeeId (Primary Key)	Employee Name, Email

SQL- Stored Procedure, Functions & Others

Q91.What is the difference between Stored Procedure and Functions?

Ans-Stored Procedure vs Function

Feature	Stored Procedure (SP)	Function (UDF – User Defined Function)
Definition	A precompiled set of SQL statements that **performs actions** in the database	A precompiled set of SQL statements that **returns a value** (scalar or table)
Return Type	May return **zero, one, or multiple values** via output parameters	Must return **a single value** or **table**
Call Method	Executed using EXEC or EXECUTE	Called inside SQL statements (SELECT, WHERE, etc.)
Side Effects	Can **modify data** (INSERT, UPDATE, DELETE)	Cannot modify database data (read-only)
Transaction Support	Supports transactions	Cannot start or commit transactions
Parameters	Input and output parameters allowed	Only input parameters
Use Case	Complex operations, batch jobs, DML operations, business logic	Calculations, reusable expressions, filtering, derived columns

Feature	Stored Procedure (SP)	Function (UDF – User Defined Function)
Example	Updating employee salary based on rules	Calculating age from birthdate

Q92-How to optimize a Stored Procedure or SQL Query?

Ans-**Query optimization** means **improving the performance of SQL queries or stored procedures** so that they execute faster and use fewer resources.

Why Optimization is Important

- Reduce **execution time**
- Minimize **CPU and memory usage**
- Improve **application performance**
- Avoid **locking and blocking** issues

Techniques to Optimize Stored Procedures / SQL Queries

Technique	Explanation	Example / Tip
1. Use Proper Indexes	Index the columns used in WHERE, JOIN, ORDER BY, GROUP BY	Create **clustered** for primary key, **non-clustered** for search columns
2. Avoid SELECT *	Only select necessary columns	SELECT Name, Salary FROM Employees instead of SELECT *
3. Use Joins Efficiently	Use proper join types (INNER JOIN, LEFT JOIN) and conditions	Avoid unnecessary joins; filter before joining
4. Filter Early	Use WHERE clause to reduce data processed	SELECT * FROM Orders WHERE OrderDate >= '2025-01-01'
5. Avoid Functions on Columns in WHERE	Functions on columns prevent index usage	Instead of WHERE YEAR(OrderDate) = 2025, use WHERE OrderDate >= '2025-01-01' AND OrderDate < '2026-01-01'
6. Use Temporary Tables / Table Variables	Break complex queries into smaller steps	Store intermediate results in #TempTable

Technique	Explanation	Example / Tip
7. Avoid Cursors if Possible	Cursors are slow, use **set-based operations**	UPDATE Employees SET Salary = Salary*1.1 WHERE DeptId=2 instead of row-by-row loop
8. Use Stored Procedure Parameters	Avoid hard-coded values; reuse execution plan	@DeptId INT instead of WHERE DeptId = 2
9. Check Execution Plan	Analyze query execution plan to find bottlenecks	Look for **table scans**, missing indexes, or high-cost operations
10. Optimize Subqueries	Replace with **JOINs** or **CTEs** if possible	Avoid correlated subqueries inside loops
11. Batch Large Updates / Inserts	Break large operations into smaller batches	TOP(1000) updates to avoid log growth and blocking
12. Avoid Unnecessary Distinct / Order By	Use only when needed	DISTINCT and ORDER BY can slow queries

Codewithindu

Example: Before & After Optimization

Before Optimization

```
SELECT *

FROM Orders

WHERE YEAR(OrderDate) = 2025

AND CustomerID IN (SELECT CustomerID FROM Customers WHERE Country='USA');
```

Problems:

- YEAR(OrderDate) prevents index usage
- Subquery can be slow

After Optimization

```
SELECT o.OrderID, o.OrderDate, o.CustomerID
```

```
FROM Orders o

INNER JOIN Customers c ON o.CustomerID = c.CustomerID

WHERE o.OrderDate >= '2025-01-01' AND o.OrderDate < '2026-01-01'

AND c.Country = 'USA';
```

Benefits:

- Index on OrderDate can be used
- Joins are efficient
- Only required columns selected

Interview Tips

1. Always **use indexes smartly**
2. **Avoid SELECT ***
3. Prefer **set-based operations** over loops
4. Check **execution plan** for table scans
5. Filter **early and efficiently**

Q93-What is a Cursor? Why to avoid them?

Ans-A **Cursor** in SQL Server is a database object that **allows row-by-row processing** of query results.

In simple words: A cursor lets you **loop through each row individually**, instead of processing all rows at once.

Why Cursors Are Used

- When you need **row-by-row operations** that cannot be done with set-based queries.
- Examples: Complex calculations, auditing, or calling procedures for each row.

Cursor Example

```
DECLARE @EmpID INT, @Salary DECIMAL(10,2)


DECLARE EmployeeCursor CURSOR FOR
```

```
SELECT EmployeeId, Salary
FROM Employees
WHERE DeptId = 2

OPEN EmployeeCursor
FETCH NEXT FROM EmployeeCursor INTO @EmpID, @Salary

WHILE @@FETCH_STATUS = 0
BEGIN
    -- Example: Give 10% bonus to each employee
    UPDATE Employees
    SET Salary = Salary * 1.1
    WHERE EmployeeId = @EmpID

    FETCH NEXT FROM EmployeeCursor INTO @EmpID, @Salary
END

CLOSE EmployeeCursor
DEALLOCATE EmployeeCursor
```

Why to Avoid Cursors

Reason	**Explanation**
Slow Performance	Cursors process **one row at a time**, which is slower than **set-based operations**

Reason	Explanation
High Memory Usage	Cursors **hold data in memory**, increasing resource usage
Locks and Blocking	Can **lock rows** for longer periods, affecting concurrency
Complexity	Code is **longer and harder to maintain** compared to set-based queries

Q94.What is CTE in SQL Server?

Ans-A **CTE (Common Table Expression)** is a **temporary, named result set** in SQL Server that you can reference **within a SELECT, INSERT, UPDATE, or DELETE statement**.

In simple words: A CTE is like a **temporary table or view** that exists only for the duration of the query.

Why Use CTE

- Simplifies **complex queries**
- Makes **recursive queries** easier
- Improves **readability** and maintainability
- Can be used to **break down large queries** into smaller, understandable pieces

Syntax

```
WITH CTE_Name (Column1, Column2, ...)
AS
(
    -- Your query here
    SELECT Column1, Column2
    FROM TableName
    WHERE Condition
)
```

```
SELECT *
FROM CTE_Name
WHERE Column1 = 'SomeValue';
```

Example 1: Simple CTE

Get all employees in Department 2:

```
WITH DeptEmployees AS
(
    SELECT EmployeeId, Name, DeptId
    FROM Employees
    WHERE DeptId = 2
)
SELECT *
FROM DeptEmployees;
```

Codewithindu

Q95. What is the difference between Delete, Truncate and Drop commands?

Ans-DELETE, TRUNCATE, and DROP in SQL Server

Feature	DELETE	TRUNCATE	DROP
Definition	Removes **specific rows** from a table based on a condition	Removes **all rows** from a table, but keeps the table structure	Deletes the **entire table or database** including structure and data
WHERE Clause	Can use WHERE to delete specific rows	Cannot use WHERE	Not applicable
Transaction Log	Logs each row deleted → **slower for large tables**	Minimal logging → **faster**	Logs table removal
Identity Column Reset	Does **not reset** identity	**Resets** identity to seed value	Not applicable

Feature	DELETE	TRUNCATE	DROP
Triggers	Activates triggers	Does **not activate** triggers	Not applicable
Rollback	Can be rolled back if within a transaction	Can be rolled back if within a transaction	Cannot rollback table structure removal once committed
Speed	Slower for large tables	Faster for large tables	Immediate removal of table
Effect on Structure	Table structure remains	Table structure remains	Table or database is removed completely

SQL Examples

1. DELETE

-- Delete specific employee

```
DELETE FROM Employees
WHERE EmployeeId = 5;
```

2. TRUNCATE

-- Remove all rows, keep table structure

```
TRUNCATE TABLE Employees;
```

3. DROP

-- Remove table completely

```
DROP TABLE Employees;
```

Real-World Analogy

Command	Analogy
DELETE	Remove specific pages from a book
TRUNCATE	Remove all pages, but keep the book
DROP	Destroy the book completely

Q96-How to get the 2nd highest salary of an employee?

Ans-Table Example: Employees

EmployeeId	Name	Salary
1	Alice	5000
2	Bob	7000
3	Charlie	6000
4	David	7000
5	Eve	4000

Using Subquery

```
SELECT MAX(Salary) AS SecondHighestSalary

FROM Employees

WHERE Salary < (SELECT MAX(Salary) FROM Employees);
```

Explanation:

- SELECT MAX(Salary) → gets the highest salary **less than the top salary**
- Simple and works in almost all SQL Server versions

Using DENSE_RANK()

```
WITH SalaryRank AS

(

    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS RankNum

    FROM Employees

)

SELECT Salary

FROM SalaryRank

WHERE RankNum = 2;
```

Explanation:

- DENSE_RANK() handles **duplicate salaries correctly**
- Returns the 2nd highest salary even if multiple employees have the highest salary

Q97.What are ACID properties?

Ans- **ACID Properties**

ACID is an acronym that defines the **key properties of a reliable database transaction** to ensure data integrity.

- **A → Atomicity**
- **C → Consistency**
- **I → Isolation**
- **D → Durability**

Atomicity

- **Definition:** A transaction is **all or nothing**. Either all operations succeed or none do.
- **Example:** Transferring $100 from Account A to Account B:
  - Debit $100 from A → Success
  - Credit $100 to B → Fail
  - Result: **Transaction rolled back**, money not deducted

Consistency

- **Definition:** A transaction must bring the database from **one valid state to another valid state**.
- **Example:** If a table requires Balance >= 0:
  - Transaction must not allow Balance < 0
  - Database remains consistent

Isolation

- **Definition:** Transactions are **isolated** from each other. One transaction's changes are **not visible to others** until committed.
- **Example:**
  - Transaction T1 updates Salary of Employee X
  - Transaction T2 querying the same row will **not see uncommitted changes** of T1

Durability

- **Definition:** Once a transaction is committed, **changes are permanent** even in case of system crash.
- **Example:**
  - You update an order status to "Shipped"
  - System crashes
  - After restart, status **remains "Shipped"**

Real-World Analogy

- **Atomicity:** Paying for items → either all items are paid or none
- **Consistency:** Bank rules → account balance cannot be negative
- **Isolation:** People at a counter → your transaction doesn't affect others until finished
- **Durability:** Receipt printed → transaction is permanent

SQL- Basics

Q98-What is the difference between DBMS and RDBMS?

Ans- DBMS vs RDBMS

Feature	DBMS (Database Management System)	RDBMS (Relational Database Management System)
Definition	Software to **store, manage, and retrieve data**	Software to **store data in tables (relations) with relationships**
Data Structure	Data stored as **files** (hierarchical, network, or flat)	Data stored in **tables (rows and columns)**
Relationships	No concept of **relationships between tables**	Tables are related using **primary key and foreign key**
Normalization	Not supported	Supported → reduces redundancy
Integrity Constraints	Limited (manual enforcement)	Supported (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK)
Data Redundancy	Higher	Lower

Feature	DBMS (Database Management System)	RDBMS (Relational Database Management System)
Examples	Microsoft Access, File System, dBase	SQL Server, Oracle, MySQL, PostgreSQL
Concurrency	Limited	Supported (multiple users)
Scalability	Smaller databases	Larger databases, enterprise-level

Real-World Analogy

- **DBMS:** A **file cabinet** where each drawer has independent files
- **RDBMS:** A **well-organized library** with tables (books), and relationships between them (author, publisher, category)

Q99What is a Constraint is SQL? What are its types.

Ans- A **Constraint** is a **rule applied on a table column** to **enforce data integrity and consistency** in the database.

In simple words: Constraints **restrict the type of data** that can be stored in a table to ensure correctness.

Types of Constraints in SQL

Constraint	Purpose	Example
PRIMARY KEY	Uniquely identifies each record in a table	EmployeeID INT PRIMARY KEY
FOREIGN KEY	Maintains referential integrity between tables	DeptID INT FOREIGN KEY REFERENCES Departments(DeptID)
UNIQUE	Ensures all values in a column are unique	Email VARCHAR(50) UNIQUE
NOT NULL	Ensures a column cannot have NULL values	Name VARCHAR(50) NOT NULL
CHECK	Ensures column values satisfy a condition	Salary DECIMAL CHECK(Salary > 0)
DEFAULT	Provides a default value if none is specified	JoinDate DATETIME DEFAULT GETDATE()

Sql Example

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Email VARCHAR(50) UNIQUE,
    DeptID INT,
    Salary DECIMAL(10,2) CHECK(Salary > 0),
    JoinDate DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
```

Explanation:

- PRIMARY KEY → EmployeeID must be unique
- NOT NULL → Name cannot be empty
- UNIQUE → Email must be unique
- CHECK → Salary must be positive
- DEFAULT → JoinDate defaults to current date if not provided
- FOREIGN KEY → DeptID must exist in Departments table

Constraints in SQL are rules applied to table columns to ensure data integrity and consistency, such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, and DEFAULT.

Q100.What is the difference between Primary key and Unique key?

Ans- Primary Key vs Unique Key

Feature	**Primary Key**	**Unique Key**
Purpose	Uniquely identifies each row in a table	Ensures all values in a column or combination of columns are unique
Null Values	Cannot have NULL	Can have **one NULL** (in SQL Server)

Feature	Primary Key	Unique Key
Number of Keys per Table	Only **one** primary key per table	Can have **multiple unique keys** per table
Index	Automatically creates a **unique clustered index** (by default)	Automatically creates a **unique non-clustered index** (by default)
Combination of Columns	Can be **single or composite**	Can also be **single or composite**
Purpose in Design	Identifies **the main identifier** for a table	Prevents **duplicate values** for other columns

SQL Example

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,        -- Primary Key
    Email VARCHAR(50) UNIQUE,          -- Unique Key
    Name VARCHAR(50) NOT NULL
);
```

Explanation:

- EmployeeID → Primary Key, must be **unique and not null**
- Email → Unique Key, must be **unique**, can be **NULL once**

Q101.What is the difference between Having clause and Where clause?

Ans- WHERE vs HAVING Clause

Feature	WHERE Clause	HAVING Clause
Purpose	Filters rows **before aggregation**	Filters groups **after aggregation**
Used With	SELECT, UPDATE, DELETE	GROUP BY and aggregate functions
Aggregate Functions	Cannot use aggregate functions (SUM, COUNT, AVG)	Can use aggregate functions

Feature	WHERE Clause	HAVING Clause
Execution	Applied **row by row**	Applied **on grouped data**
Example Usage	Filter employees with Salary > 5000	Filter departments where SUM(Salary) > 20000

SQL Examples

1️⃣WHERE Clause

-- Get employees with salary > 5000

SELECT EmployeeID, Name, Salary

FROM Employees

WHERE Salary > 5000;

2️⃣HAVING Clause

-- Get departments where total salary > 20000

SELECT DeptID, SUM(Salary) AS TotalSalary

FROM Employees

GROUP BY DeptID

HAVING SUM(Salary) > 20000;

Explanation:

- WHERE filters **individual rows** before aggregation
- HAVING filters **aggregated results** after GROUP BY

Q102.What is Sub query or Nested query or Inner query in SQL?

Ans- A **Subquery** (also called **Nested Query** or **Inner Query**) is a **query inside another query**.

- The **inner query** runs first, and its result is used by the **outer query**.
- Helps to **break complex queries into smaller, manageable parts**.

In simple words: A **query inside a query** to filter or calculate data dynamically.

Types of Subqueries

1. **Single-row Subquery** – returns **one row**
2. **Multiple-row Subquery** – returns **multiple rows**
3. **Correlated Subquery** – inner query **depends on outer query**

SQL Examples

1️⃣ Single-row Subquery

Get employees who earn the **highest salary**:

```
SELECT Name, Salary

FROM Employees

WHERE Salary = (SELECT MAX(Salary) FROM Employees);
```

- Inner query: (SELECT MAX(Salary) FROM Employees)
- Outer query: selects employees with that salary

2️⃣ Multiple-row Subquery

Get employees who belong to departments 101 or 102:

```
SELECT Name, DeptID

FROM Employees

WHERE DeptID IN (SELECT DeptID FROM Departments WHERE DeptName IN ('HR','IT'));
```

3️⃣ Correlated Subquery

Get employees earning **more than average salary in their department**:

```
SELECT Name, Salary, DeptID

FROM Employees e1

WHERE Salary > (

    SELECT AVG(Salary)
```

```
    FROM Employees e2
    WHERE e1.DeptID = e2.DeptID
);
```

- Inner query uses e1.DeptID from the outer query → **correlated**

A Subquery or Nested Query is a query inside another query where the inner query executes first and provides results for the outer query, used to simplify complex SQL operations.

Q103.What is a Foreign Key?

Ans-A **Foreign Key** is a column (or group of columns) in one table that **creates a relationship** between two tables.

- It **refers to the Primary Key** of another table.
- It ensures **data consistency** and **prevents invalid data**.
- It maintains **referential integrity**.

Simple words:
A **Foreign Key connects two tables** like a link.
It makes sure the value exists in the parent table.

Example

Table: Department

```
CREATE TABLE Department (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);
```

Table: Employee

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
```

```
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

What happens here

- DeptID in **Employee** must exist in Department.DeptID
- You **cannot add an employee** with a department that does not exist
- This maintains **record accuracy**

A Foreign Key is a column in a table that creates a relationship with the primary key of another table and ensures referential integrity by preventing invalid data.

Q104.Difference between Web.config and appsettings.json

Ans-1. **Platform**

Feature	**Web.config**	**appsettings.json**
Used in	ASP.NET (Full .NET Framework)	ASP.NET Core
Technology	XML-based configuration	JSON-based configuration

👉 **Web.config** = Old ASP.NET

👉 **appsettings.json** = New ASP.NET Core

2. File Format

- **Web.config → XML format**

```
<configuration>
 <appSettings>
  <add key="AppName" value="MyApp" />
 </appSettings>
</configuration>
```

- **appsettings.json → JSON format**

```
{
```

```
  "AppSettings": {
    "AppName": "MyApp"
  }
}
```

3. Flexibility

Web.config	**appsettings.json**
Harder to read	Easy to read
XML is verbose	JSON is lightweight
Limited environment support	Full environment support (Dev, QA, Prod)

4. Environment Support

ASP.NET (Web.config)

- Typically uses **Web.Debug.config**, **Web.Release.config**
- Uses **XML transformation**

ASP.NET Core (appsettings.json)

- Multiple environment files:
  - **appsettings.Development.json**
  - **appsettings.Staging.json**
  - **appsettings.Production.json**

This makes ASP.NET Core more flexible.

5. Reloading Changes

Web.config	**appsettings.json**
Change reloads the application (IIS restarts AppDomain)	Supports **automatic reload** without restarting
Not developer-friendly	Very developer-friendly

6. Dependency Injection Support

Web.config

- No direct DI support
- Access using ConfigurationManager

appsettings.json

- Built-in integration with Dependency Injection
- Access using IOptions<T>

Example:

```
builder.Services.Configure<MySettings>(builder.Configuration.GetSection("MySettings"));
```

7. Storing Connection Strings

Both support it, but Core uses a cleaner format.

Web.config:

```
<connectionStrings>
 <add name="Default" connectionString="..." providerName="System.Data.SqlClient" />
</connectionStrings>
```

appsettings.json:

```
{
  "ConnectionStrings": {
    "Default": "..."
  }
}
```

Q105.Difference Between Function and Stored Procedure?

Ans-

Function

Used to **perform calculations** and **return a value**.

Stored Procedure

Used to **perform operations** like insert, update, delete, select.

Feature	Function (UDF)	Stored Procedure (SP)
Purpose	Used for calculations and returning a value	Used for performing business logic (CRUD)
Return Type	Must return a value (scalar/table)	Optional – may or may not return a value
Return Keyword	Required	Not required
Parameters	Only **IN** parameters	IN, OUT, and INOUT parameters
Usage in SELECT	✓ Yes (can be used in SELECT, WHERE, JOIN)	✕ No
DML Operations (INSERT/UPDATE/DELETE)	✕ Not allowed	✓ Allowed
Error Handling (TRY...CATCH)	✕ Not supported	✓ Supported
Transactions (BEGIN TRAN)	✕ Not allowed	✓ Allowed
Performance	Faster for calculations	Better for complex operations
Calling Style	Used like a value: SELECT dbo.Func()	Executed: EXEC ProcedureName
Stored in	Stored as database object	Stored as database object
Compilation	Compiled every time	Compiled and cached

Codewithindu

Feature	Function (UDF)	Stored Procedure (SP)
Trigger calling	Can be called from trigger	Can be executed from trigger

Q106.How to Explain Database Design Without Revealing Your Domain

Ans-When you work under NDA or cannot reveal your project domain (healthcare, finance, government, etc.), you must describe **your database design approach** in a **generic, domain-independent way**.

I cannot disclose the exact domain due to NDA, but I can explain my database design process in a domain-neutral, technical way."

1️⃣ Requirement Understanding

- Identify the entities (objects)
- Identify their attributes (fields)
- Identify relationships between entities

Example:
Entities like *User*, *Role*, *Document*, *Transaction* (generic words, no domain leak)

2️⃣ Entity Identification & Normalization

You explain your process, NOT the real domain.

Remove duplicate data
Apply normalization up to **3rd Normal Form (3NF)**
Separate lookup/master tables
Ensure minimal redundancy

Example:

- Separate Users, UserRoles
- Separate Products, ProductCategories

This shows your skill without exposing the real domain.

3️⃣ Relationship Design

Use generic relationship terms:

- **One-to-One** → Profile & User
- **One-to-Many** → User & Orders
- **Many-to-Many** → Users & Roles (junction table)

Generic ER Diagram wording (safe):

User ---< Orders

Product ---< OrderDetails

Role >---< User (Mapping table)

No domain exposed.

4️⃣ Constraints and Indexing

Explain technical details instead of real data:

- Primary Keys, Foreign Keys
- Unique Constraints
- Check Constraints
- Clustered and Non-Clustered Indexes
- Composite Indexes for performance

Example:
"I added a non-clustered index on Email for fast login checks."

5️⃣ Query Optimization

Focus on techniques, not domain:

- Avoid SELECT *
- Use proper JOINs
- Use stored procedures
- Add indexes on frequent columns
- Avoid cursor, use CTE instead

6.**Security Considerations**

Keep it generic:

- Encryption for sensitive fields
- Role-based access
- Soft delete instead of physical delete

Q107. What is Revisit Rule?

Ans-**A practice where developers should revisit (recheck) the code, logic, or requirement after writing it** to ensure correctness, quality, and maintainability.

It is similar to **code review**, **refactoring**, and **validation**.

or

Revisit Rule means reviewing your own work (code, query, design, logic, or requirements) at least once before finalizing it, to catch mistakes early and improve quality."

It helps in:

- Finding hidden bugs
- Improving code readability
- Applying best practices (SOLID, DRY, KISS)
- Optimizing logic
- Ensuring alignment with requirements

Q108. What are Triggers and types of triggers?

Ans-A **Trigger** is a special type of stored procedure that **automatically executes** when an event occurs in a table.

Trigger = Automatic Action on Table Events

Trigger fires when you perform:

- INSERT
- UPDATE
- DELETE

Triggers help with:

- Auditing (who changed data?)
- Enforcing business rules
- Maintaining history
- Validation
- Cascading updates

A trigger is a database object that automatically executes in response to INSERT, UPDATE, or DELETE operations on a table.

Simple Real-World Example

Imagine a **banking system**:

Whenever money is withdrawn:

Automatically record the transaction in **TransactionLog** table.

A trigger can do this automatically.

Types of Triggers

SQL Server has **two main types**:

1. AFTER Trigger (MOST COMMON)

Fires **after** an INSERT, UPDATE, or DELETE operation is completed.

Used for:

- Maintaining logs
- Sending notifications
- Enforcing business rules

Example:

```
CREATE TRIGGER trgAfterInsertEmployee
ON Employees
AFTER INSERT
AS
BEGIN
```

```
    INSERT INTO LogTable (Message)
    VALUES ('New Employee Added')
END
```

2. INSTEAD OF Trigger

Executes **instead of** the INSERT/UPDATE/DELETE operation.

Used when:

- You want to **control** or **override** how data is inserted/updated.
- Useful for **views** that cannot directly update multiple tables.

Example:

```
CREATE TRIGGER trgInsteadOfDelete
ON Employees
INSTEAD OF DELETE
AS
BEGIN
    PRINT 'Delete operation is not allowed!'
END
```

Codewithindu

Q109.What is Azure DevOps CI/CD?

Ans-Azure DevOps CI/CD is Microsoft's service for **automating the build, testing, and deployment** of applications.

- **CI (Continuous Integration)** → Automatically builds and tests the code whenever developers push changes.
- **CD (Continuous Deployment/Delivery)** → Automatically deploys the tested code to environments like Dev, QA, UAT, Production.

Azure DevOps CI/CD is the automated pipeline that takes code from source control (like Git), builds it, tests it, and deploys it to target environments using Azure Pipelines.

Real-World Example

Suppose you are building a **.NET Web API**.

When you push your code to GitHub or Azure Repos:

1. **CI Pipeline starts**
   ✓ Builds code
   ✓ Runs unit tests
   ✓ Publishes artifacts
2. **CD Pipeline starts**
   ✓ Takes artifact
   ✓ Deploys to Dev → QA → Production

Automation = **No manual deployment, fewer errors, faster releases.**

CI/CD Pipeline Diagram (Simple & Easy to Remember)

Developer → Push Code → CI Pipeline → Build → Test → Artifact → CD Pipeline → Deploy → Dev/QA/Prod

What is CI (Continuous Integration)?

Automatically builds and tests code whenever a developer commits changes.

Goal

- Detect bugs early
- Avoid integration issues
- Maintain consistent builds

What CI does in Azure DevOps

- Restore NuGet packages
- Build the application
- Run automated tests
- Create build artifacts

What is CD (Continuous Delivery/Deployment)?

Automatically delivers the built application to servers or cloud environments.

Continuous Delivery

Manual approval is required before deploying to Production.

Continuous Deployment

No manual approval — fully automated.

What CD does

- Deploy to Azure App Service
- Deploy to Kubernetes
- Deploy to IIS/VMs
- Deploy to SQL Databases

Azure DevOps CI/CD Components

1️⃣ Azure Repos – Source control (Git)

2️⃣ Azure Pipelines – CI/CD automation

3️⃣ Azure Artifacts – Packages storage

4️⃣ Azure Test Plans – Manual + automated tests

5️⃣ Azure Boards – Work management (Agile, Scrum)

How CI/CD Pipeline Works in Azure DevOps?

Step 1: Developer Pushes Code

To GitHub/Azure Repos.

Step 2: CI Pipeline Runs (azure-pipelines.yml)

```
trigger:
  - main

pool:
  vmImage: 'windows-latest'

steps:
```

```
- task: DotNetCoreCLI@2
  inputs:
    command: 'build'
    projects: '**/*.csproj'
```

Step 3: Artifact Generated

Build output is saved.

Step 4: CD Pipeline Runs

Deploys to:

- Azure App Service
- Virtual Machine
- Kubernetes (AKS)
- Docker Containers

Q110. **What is Remote Debugging?**

Remote Debugging means **debugging an application that is running on another machine**, not on your local computer.

In simple words:

You debug your code on a server or remote system from your own computer.

Why Do We Need Remote Debugging?

Sometimes your application runs correctly on your local system but crashes or behaves differently on:

- A **production server**
- A **test server**
- A **client's machine**
- A **cloud environment** (Azure, AWS)
- A **Docker container**

In such cases, you cannot copy the entire server setup to your local machine.

So you use **Remote Debugging** to connect to that remote environment and debug the problem.

How Remote Debugging Works (Simple Explanation)

1. Your application runs on a **remote machine** (server).
2. You **attach your debugger** (e.g., Visual Studio, IntelliJ, VS Code) from your local system.
3. A **remote debugging agent** runs on the server.
4. Your system connects to that agent through the network.
5. You can:
    - Set breakpoints
    - Check variable values
    - Step into code
    - Catch exceptions

Just like debugging locally — but happening remotely!

Example (In .NET using Visual Studio)

Suppose:

Your API is running on a **Windows server**.

Steps:

1. Install **Remote Debugging Tool** on the server.
2. Run the app on the server normally.
3. From Visual Studio (your PC):
   → Go to **Debug → Attach to Process**
   → Select remote machine
   → Choose your API process
4. Set breakpoints and debug.

That’s it — you're reading server code from your own laptop!

Q111.What is Dependency Injection?

Ans-Dependency Injection (DI) is a **design pattern** in which a class receives (or is injected with) the objects it depends on **from outside**, instead of creating them inside the class.

In simple words:

DI means *giving* the object what it needs instead of the object *making* it by itself.

Why Do We Use Dependency Injection?

Because it helps in:

Reducing tight coupling
Increasing code reusability
Improving testability
Cleaner, modular architecture
Easier maintenance

Simple Real-Life Example

Imagine you switch a bulb in your home.

- You **do not create** electricity inside the bulb.
- Electricity is **injected** from outside (via wire).

Similarly, in software:

- A class should **not create** its own dependencies.
- They should be **provided** from outside.

This is Dependency Injection.

Simple Technical Example (Without DI ✕)

```
public class StudentService
{
    private StudentRepository _repo;
```

```
    public StudentService()
    {
        _repo = new StudentRepository(); // Tight Coupling ✕
    }

    public void Save() => _repo.Add();
}
```

Problem:

- StudentService is tightly coupled with StudentRepository.
- Hard to test (cannot mock repo).
- Hard to change implementation.

With Dependency Injection (Correct ✓)

```
public class StudentService
{
    private readonly IStudentRepository _repo;

    public StudentService(IStudentRepository repo)
    {
        _repo = repo; // Dependency Injected ✓
    }

    public void Save() => _repo.Add();
}
```

Benefits:

- Loosely coupled

Codewithindu

- Flexible
- Easy to mock for testing
- Can swap repository implementations

How DI Works in .NET Core

We register services in Program.cs:

```
services.AddScoped<IStudentRepository, StudentRepository>();
services.AddScoped<StudentService>();
```

The framework automatically injects dependencies when needed.

Dependency Injection in Angular

```
export class StudentService {
  constructor(private http: HttpClient) { }
}
```

Angular automatically injects HttpClient.

Types of Dependency Injection

1. **Constructor Injection** (most common)
    - Dependencies passed through constructor
    - Example: new Service(repo)
2. **Property Injection**
    - Dependencies assigned through properties
3. **Method Injection**
    - Dependencies passed as method parameters

Simple Interview Answer

Dependency Injection is a design pattern used to achieve loose coupling between classes. Instead of a class creating the objects it depends on, those objects are provided from outside. This makes code flexible, testable, and easier to maintain.

Q112. What is Database Design?

Ans-Database Design is the process of **organizing data** in a structured way so that it can be stored, retrieved, and managed efficiently in a database.

Simple Definition:

Database Design means planning how data will be stored in tables and how those tables will be linked.

Purpose of Database Design

A good database design ensures:

Fast and efficient queries
No duplicate data
Consistency and accuracy
Easy maintenance
Reliable data relationships
Scalable performance

Why is Database Design Important?

Because bad design causes:

❌ duplicate data
❌ slow queries
❌ data inconsistency
❌ complex logic
❌ future problems

Steps in Database Design

1️⃣ Requirement Gathering

Understand what the system will store.
Example: For a School system → Students, Teachers, Classes.

2️⃣ Identify Entities

Entities = Objects in the system.
Example:

- Student
- Teacher
- Class
- Address

3️⃣ Define Attributes

Attributes = Details of each entity.
Example:
Student → Name, Age, Email, Phone, ClassName

4️⃣ Create Tables

Convert entities into tables.

5️⃣ Define Relationships

Use **Primary Key** and **Foreign Key**.

Example:

- One Class has many Students → **1-to-Many**
- One Student has many Addresses → **1-to-Many**

6️⃣ Apply Normalization

To avoid duplicate and unnecessary data.

Example:
Instead of storing Address inside Student table →
Create a separate **Address** tabl

7️⃣ Create ER Diagram

Shows how tables connect with each other.

Simple Example

School Management Database Design

1. Student Table

StudentId (PK) Name Age ClassId (FK)

2. Class Table

ClassId (PK) ClassName

3. Address Table

AddressId (PK) StudentId (FK) City ZipCode

Relationships

- **Class → Student** → One-to-Many
- **Student → Address** → One-to-Many

Thus, the design is clean, normalized, and easy to maintain.

Q113.Difference Between Class and Structure in C#

Ans-

Feature	**Class**	**Structure (Struct)**
Type	Reference Type	Value Type
Memory Allocation	Stored on **Heap**	Stored on **Stack**
Default Behavior	Passed by **reference**	Passed by **value** (copy is created)
Can Have Inheritance?	✓ Yes (supports inheritance)	✕ No (cannot inherit classes/structs)
Constructor	Can have **default and parameterized** constructors	Cannot have **default** constructor; only parameterized allowed
Destructor	✓ Allowed	✕ Not allowed
Performance	Slightly slower due to heap allocation	Faster for small data because stored on stack

Feature	Class	Structure (Struct)
Use Case	Large data model, complex behavior	Small, lightweight data (like coordinates, points)
Null Allowed?	✓ Yes	✗ No (unless declared as nullable struct: int?)
Members	Can have events, methods, properties	Can also have all, but no inheritance
Polymorphism	✓ Supported	✗ Not supported

Simple Explanation

Class → Reference Type

- When you pass a class object to a function → actual object is referenced.
- Changes inside the function will affect the original object.

Struct → Value Type

- When you pass a struct → a copy is made.
- Changing the copy does not affect the original.

Simple Real-Life Example

Class Example (Student Class)

```
public class Student
{
   public string Name;
   public int Age;
}
```

Struct Example (Point Struct)

```
public struct Point
{
```

```
    public int X;
    public int Y;
}
```

When to Use What

Use Class

- When the model is complex
- When you need inheritance, OOP features
- When object size is large
- When you want reference behavior

Examples:
Student, Employee, Product, Order, Vehicle

Use Struct

- When data is small, simple, and represents a single value/entity
- When you need performance on small objects
- Used mostly in **coordinates, colors, measurements**

Examples:
Point, Coordinate, RGB Color, Money Struct

Q114.What is difference between Overloading and Overriding?

Ans-

Feature	**Method Overloading**	**Method Overriding**
Definition	Same method name with different parameters in the same class	Same method name and parameters in different classes (inheritance)
Type of Polymorphism	Compile-time Polymorphism	Run-time Polymorphism

Feature	Method Overloading	Method Overriding
Where Implemented	Within the same class	Between Parent & Child classes
Parameters	Must be different (count, type, order)	Must be exactly same as parent method
Return Type	Can be same or different	Must be same as parent (in C#)
Inheritance Required	✕ Not required	✓ Required
Keywords Used	None	virtual, override, abstract
Binding Time	Early binding	Late binding
Purpose	Increases readability & flexibility	Changes or extends parent class behavior
Example	Add(int a, int b) and Add(double x, double y)	Parent: Speak() → Child: override Speak()

Q115.Difference between Interface and Abstract ?

Ans-An **Interface** is a contract that **only defines what a class must do**, but **not how**.

Contains **methods without implementation** (before C# 8)
Supports **multiple inheritance**
Used when you want **common behavior** across unrelated classes

2. Abstract Class — Definition

An **Abstract Class** is a partially implemented class that can have:
Abstract methods (no body)
Normal methods (with body)
Fields, constructors, properties

Used when you want **common functionality** shared across related classes.

Feature	Interface	Abstract Class
Purpose	Specifies a “contract”	Serves as a base class with shared logic
Methods	Only abstract methods (before C# 8)	Can have abstract + non-abstract methods
Implementation	No method body (except default interface methods)	Can contain full method implementation
Variables/Fields	No fields allowed	Can have fields, properties, constants
Constructor	✗ No constructor	✓ Can have constructor
Multiple Inheritance	✓ Supports multiple inheritance	✗ Does not support multiple inheritance
Access Modifiers	All members public by default	Can have private/public/protected
When to Use?	When classes share only behavior	When classes share behavior + common logic
Example Use	Logging, payment methods, repository pattern	Animal base class, vehicle base class

Simple Real-World Example

Interface Example – Contract

```
public interface IPayment
{
    void Pay();
}
```

```
public class CreditCard : IPayment
{
    public void Pay() => Console.WriteLine("Paid using Credit Card");
}

public class UPI : IPayment
{
    public void Pay() => Console.WriteLine("Paid using UPI");
}
```

➞ All payment methods must implement **Pay()**.
➞ No common logic shared — so **interface** is best.

Abstract Class Example – Common Logic

Codewithindu

```
public abstract class Animal
{
    public void Eat()  // Shared logic
    {
        Console.WriteLine("Animal Eating...");
    }

    public abstract void Sound();  // Different for each animal
}

public class Dog : Animal
{
    public override void Sound()
```

```
    {
        Console.WriteLine("Dog Barks");
    }
}
```

➞ All animals **eat the same**, but have **different sounds**.
➞ Perfect for **abstract class**.

Q116.What is out and ref argument in C#?

Ans-In C#, **out** and **ref** are used to **pass parameters by reference** instead of by value.

This means:

- The method can **modify the original variable**.
- The changes are reflected **outside the method**.

1. ref Keyword — Definition

- ref **requires** the variable to be **initialized before passing**.
- Used when you want to **read and modify** the value.

Example

```
void Update(ref int number)
{
    number = number + 10;
}

int value = 5;
Update(ref value);
Console.WriteLine(value);  // Output: 15
```

Because we passed ref, the original variable value gets updated.

2. out Keyword — Definition

- out **does NOT require initialization** before passing.
- But the method **must assign a value** before returning.
- Used when you want a method to **return multiple values**.

Example

```
void GetDetails(out int age, out string name)
{
   age = 25;
   name = "John";
}

int a;
string n;
GetDetails(out a, out n);

Console.WriteLine(a); // 25
Console.WriteLine(n); // John
```

Method must set values for out variables.

Difference Between ref and out

Feature	**ref**	**out**
Must initialize before passing?	✓ Yes	✗ No
Must assign value inside method?	✗ No	✓ Yes
Purpose	Modify existing value	Return multiple values

Feature	ref	out
Read inside method?	✓ Yes (already has a value)	✗ No (no guarantee it has value)
Example usage	Update a variable	Methods that return multiple outputs

Q117. **What is a Testing Framework?**

A **Testing Framework** is a **set of guidelines, tools, and libraries** that helps developers **write, organize, and execute tests** efficiently and consistently.

It provides:

- A structured way to write tests
- Automation for running tests
- Reporting and logging test results

Purpose of a Testing Framework

- Ensures **software quality**
- Detects **bugs early**
- Enables **automated testing** (unit, integration, UI)
- Supports **reusable test code**

Types of Testing Frameworks

Type	Purpose	Example
Unit Testing Framework	Test individual units (methods/functions)	NUnit, MSTest, xUnit
Integration Testing Framework	Test combined modules or database interactions	SpecFlow, Postman, REST-Assured
UI Testing Framework	Automate UI interactions	Selenium, Cypress, Playwright

Type	Purpose	Example
Behavior Driven Framework (BDD)	Test based on behavior/scenarios	Cucumber, SpecFlow
Performance Testing Framework	Test system speed & load	JMeter, LoadRunner

Q118.Diiference between Unit Testing And Integrating Testing?

Ans-**Unit Testing vs Integration Testing**

1. Unit Testing

Unit Testing is the process of testing **individual components or functions** of the software in isolation to ensure they work as expected.

Purpose:

- Verify correctness of a single method or class
- Detect bugs early in development
- Facilitate code refactoring

Key Points:

- Smallest testable part of the application
- Usually automated
- Does **not** involve database, network, or external systems

Example:
Testing a Calculator class method:

```
using NUnit.Framework;


[TestFixture]
public class CalculatorTests
{
```

```
    [Test]
    public void Add_WhenCalled_ReturnsSum()
    {
        Calculator calc = new Calculator();
        int result = calc.Add(5, 3);
        Assert.AreEqual(8, result);  // Check if sum is correct
    }
}
```

Real-world example:

- Verify Login(username, password) method returns true for valid credentials.

2. Integration Testing

Integration Testing checks whether **different modules or components work together** correctly.

Purpose:

- Verify interaction between modules or external systems
- Detect issues in communication or data flow

Key Points:

- Involves multiple units/modules
- May involve database, API, or file system
- Often done after unit testing

Example:

Testing OrderService that saves orders to a database:

```
[Test]
public void PlaceOrder_WhenCalled_SavesOrderToDatabase()
{
```

```
    var orderService = new OrderService(new OrderRepository());
    var order = new Order { Id = 1, Product = "Book" };

    orderService.PlaceOrder(order);

    var savedOrder = orderService.GetOrderById(1);
    Assert.IsNotNull(savedOrder);  // Check if order was saved
}
```

Real-world example:

- Check if Login() method correctly interacts with **database** and **returns user details**.

Key Differences Table

Feature	**Unit Testing**	**Integration Testing**
Focus	Individual units (methods/classes)	Interaction between multiple modules
Scope	Narrow	Broad
Dependencies	Isolated (no DB/API)	Depends on DB, API, or other modules
Execution Time	Fast	Slower
Purpose	Verify correctness of a unit	Verify modules work together
Example	Testing Add(a,b) method	Testing OrderService.PlaceOrder() with DB

Codewithindu